

ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE

Faculté Informatique et Communications

Section Data Science

Thèse de master

DOM-Graph RAG Visuel

Tuilage guidé par la structure du document
pour le RAG visuel

Alexis Firome

Courriel : alexis.firome@epfl.ch

Adresse : 27 ter boulevard Diderot, 75012 Paris

Superviseur académique EPFL Pr. Amir Zamir

Laboratoire d'accueil Devoteam DRI

Encadrement Dr. Valentin Noël

Date de soumission 09-09-2026

Déclaration

Je déclare que ce rapport présente le travail que j'ai personnellement réalisé dans le cadre de mon projet de master. Les contributions de tiers, les résultats issus de la littérature et les outils logiciels réutilisés sont explicitement identifiés et référencés. Le code source du système décrit dans ce rapport est disponible en accès ouvert.

Une version antérieure et condensée des travaux décrits aux chapitres [2](#) et [3](#) a été soumise sous forme d'article court à une conférence en traitement automatique des langues ; le présent rapport en constitue la version longue, complétée du contexte scientifique (chapitre [1](#)), du détail d'implémentation et de l'analyse critique des résultats.

Paris, le 09-09-2026

Alexis Firome

signature

Remerciements

[Section à personnaliser avant soumission.]

Je remercie [nom du superviseur] pour l'encadrement de ce projet, ainsi que [laboratoire / personnes] pour les discussions techniques et l'accès aux ressources de calcul qui ont rendu possible la partie expérimentale de ce travail. Les erreurs et les choix discutables qui subsistent dans ce rapport sont les miens.

Table des matières

Introduction	3
Contexte	3
Le problème	3
Contribution personnelle	4
Objectifs et contributions	5
Résultats principaux	5
Organisation du rapport	6
1 État de l’art	7
1.1 Génération augmentée par récupération et découpage	7
1.1.1 Le biais de taille de vivier	8
1.2 Documents visuellement riches et prétraitement	8
1.3 Modèles vision-langage et RAG visuel	9
1.3.1 PixelRAG et le découpage aveugle	9
1.4 RAG structuré par graphe	9
1.4.1 MAGE-RAG	10
1.5 Apprentissage contrastif et adaptation efficace	10
1.5.1 InfoNCE et la qualité des négatifs	10
1.5.2 Adaptation efficace d’un modèle génératif en encodeur	10
1.6 Synthèse et positionnement	10
2 Méthodologie	13
2.1 Préliminaires géométriques : le DOM et le moteur de rendu	13
2.1.1 Du balisage à la mise en page, et le modèle de boîtes	14
2.1.2 Éléments flottants : l’écart entre boîte de bloc et lignes réelles	14
2.1.3 Les deux API de géométrie et le repère absolu	15
2.1.4 Ordre du DOM et ordre visuel	16
2.2 Rendu de page et extraction géométrique	16
2.2.1 Rendu sans interface graphique	16
2.2.2 Retrait du bruit d’interface	16
2.2.3 Extraction des boîtes englobantes	17
2.2.4 Mesure serrée des balises de texte coulant	17
2.3 Tuilage adaptatif guidé par le DOM	17
2.3.1 Élagage des éléments imbriqués	18
2.3.2 Résolution des chevauchements partiels	18
2.3.3 Les trois règles de tuilage	18
2.3.4 Modèle de coût : tokenisation visuelle et effet du tuilage	20
2.3.5 Découpage en sections des pages démesurées	21
2.4 Construction du graphe multigranulaire	22
2.4.1 Ordre de lecture spatial	22
2.4.2 Voisinage de mise en page	23
2.4.3 Arbre des sections	23
2.4.4 Voisinage sémantique	23
2.4.5 Relations inter-documents	24
2.4.6 Graphe de corpus multi-pages	24

2.4.7	Rendu structuré pour le lecteur	24
2.5	Entraînement contrastif du lecteur	25
2.5.1	Génération synthétique de paires question-tuile	25
2.5.2	Minage de négatifs difficiles	25
2.5.3	Du modèle génératif à l'encodeur	26
2.5.4	Adaptation LoRA sur la tour de vision et le décodeur	27
2.5.5	La perte contrastive	27
2.5.6	Contraintes mémoire	28
2.5.7	Partitionnement pour l'évaluation	28
2.6	Contrôleur de preuves en ligne	28
2.6.1	Machine à états	28
2.6.2	Politique <i>best-first</i> budgétée	29
2.6.3	Expansion inter-documents	30
2.6.4	Score de pertinence et point d'extension	30
2.6.5	Lecture des tuiles retenues	31
2.7	Implémentation et ingénierie	31
2.7.1	Architecture modulaire	31
2.7.2	Reproductibilité des appels au modèle	31
2.7.3	Sauvegarde par unité de travail	32
2.7.4	Concurrence	32
2.7.5	Tests	32
2.8	Protocole expérimental	32
2.8.1	Construction du jeu de données synthétique	32
2.8.2	Reproduction de la référence en grille	33
2.8.3	Ablation de résolution	33
2.8.4	Questions multi-sauts issues de HotpotQA	33
3	Résultats et limites	35
3.1	Jeu de données	35
3.2	Le tuilage guidé par le DOM double la trouvabilité, à moindre coût	35
3.2.1	Protocole	35
3.2.2	Résultat 1 : le taux d'attachement	36
3.2.3	Résultat 2 : le budget de pixels	36
3.3	L'entraînement contrastif du lecteur fait presque doubler le Recall@1	37
3.3.1	Protocole	37
3.3.2	Résultat et analyse	37
3.4	Composants en aval, coûts et limites : synthèse	38
	Conclusion	41
	Rappel de la thèse	41
	Ce que le travail établit	41
	Le verrou identifié	42
	Enseignements méthodologiques	43
	Perspectives	43
	Mot de la fin	44
A	Résultats complémentaires	45
A.1	Évaluation A, complément : protocole et classement	45
A.1.1	Deux contraintes de protocole	45
A.1.2	Résultat 3 : la qualité de classement	45
A.1.3	Pourquoi le texte plat reste compétitif	45

A.2	Évaluation C : contrôleur budgété et sélection top- k statique	46
A.2.1	Protocole	46
A.2.2	Diagnostic, correctif et résultat	47
A.2.3	Analyse	48
A.3	Évaluation D : le système assemblé	48
A.3.1	Protocole et résultat	48
A.3.2	Taxonomie des erreurs et analyse	49
A.3.3	Sous-rappel de preuves plutôt que fabulation	49
A.4	Coûts de calcul	50
A.5	Limites	50
A.6	Directions de travail	52
B	Hyperparamètres	53
C	Invites utilisées	55
C.1	Génération de paires question-réponse	55
C.2	Instruction d'encodage PromptEOL	55
C.3	Juge de faux négatifs	55
C.4	Lecture d'une tuile ouverte	56
C.5	Synthèse de la réponse finale	56
D	Organisation du dépôt	57
D.1	Modules du pipeline	57
D.2	Scripts d'expérimentation	58
D.3	Tests	58
E	Protocole de reproduction	59
E.1	Installation	59
E.2	Vérification de l'installation	59
E.3	Reproduction des évaluations	59
E.4	Application de démonstration	59
E.5	Notes de reproductibilité	60
F	Considérations éthiques	61
F.1	Accès aux données	61
F.2	Licences	61
F.3	Statut des données synthétiques	61
F.4	Aptitude au déploiement	61
	Bibliographie	63

Introduction

Contexte

Les grands modèles de langue ont une mémoire figée à la date de leur entraînement et une propension bien documentée à produire des affirmations plausibles mais fausses lorsqu’on les interroge sur des faits qu’ils n’ont pas mémorisés. La génération augmentée par récupération [23] répond à ces deux limites par la même mécanique : avant de répondre, le système va chercher dans une base documentaire les passages pertinents pour la question, puis les fournit au modèle génératif comme contexte. Le modèle n’a plus à savoir, il a à lire. Cette architecture est devenue le mode de déploiement dominant des modèles de langue en environnement professionnel, et une littérature abondante en a exploré les variantes [13].

Le maillon faible de cette chaîne est rarement le générateur. C’est le prétraitement documentaire. Un pipeline RAG classique commence par convertir chaque document en texte brut : analyse syntaxique, extraction du flux textuel, reconnaissance optique de caractères sur un document numérisé. Cette conversion est une projection destructive. Un tableau devient une suite de mots dont on ne sait plus quelle cellule appartenait à quelle ligne. Une légende se retrouve orpheline de sa figure. Un encadré latéral s’intercale au milieu d’un paragraphe qu’il ne concernait pas. Cette perte d’information n’est pas marginale sur des documents que l’on qualifie de visuellement riches comme des pages web, des rapports financiers, des notices techniques ou des articles encyclopédiques. La perte d’information porte précisément sur ce que la mise en page encodait.

Deux réponses distinctes coexistent dans la littérature récente, et ce travail ci se situe exactement à leur intersection. La première rend l’extraction inutile : puisque les modèles vision-langage savent lire une image de document, autant leur donner l’image, une page rendue, découpée en imagerie indexées par un encodeur multimodal, la récupération s’opérant directement dans l’espace visuel des pixels [10, 25]. PixelRAG [37] porte cette idée à l’échelle du web et rapporte qu’elle égale ou dépasse la récupération textuelle sur les documents longs et fortement mis en page ; mais elle découpe la page rendue sur une grille de hauteur fixe, indifférente à ce qu’elle traverse. La seconde structure les preuves plutôt que de les traiter comme un sac plat d’unités indépendantes : MAGE-RAG [45] organise un document déjà segmenté en un graphe multigranulaire qu’un contrôleur budgété parcourt à la requête, plutôt que de présenter au modèle lecteur la totalité du document ou une sélection figée.

Ces deux travaux sont complémentaires et laissent la même pièce sur la table. PixelRAG établit que des tuiles de pixels lues par un modèle vision-langage constituent une unité de récupération viable, mais les découpe à l’aveugle. MAGE-RAG établit qu’un contrôleur de graphe budgété surpasse une sélection top- k statique, mais suppose les éléments du document déjà constitués, en amont de son propre pipeline. Ce travail assemble les deux par l’artefact que PixelRAG jette et que MAGE-RAG n’a jamais eu à construire : le DOM de la page rendue.

Le problème

Ce changement de représentation laisse intacte une décision de conception sur la manière dont on doit découper la page. Un modèle vision-langage ne peut pas ingérer une capture d’écran pleine page d’un article encyclopédique de 46 000 pixels de hauteur, et de toute façon une unité de récupération doit être suffisamment fine pour qu’un score de pertinence signifie

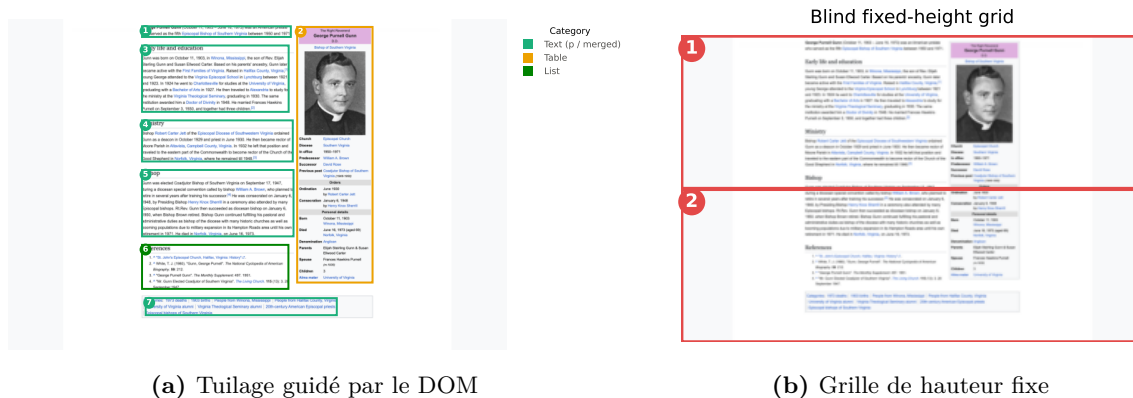


Figure 1 – La même page Wikipédia rendue (en.wikipedia.org/wiki/George_P._Gunn), découpée de deux manières. (a) Tuilage guidé par le DOM (section 2.3) : (b) Grille aveugle de hauteur fixe selon PixelRAG

quelque chose. Il faut donc découper la page en plusieurs captures. PixelRAG découpe sur une grille de hauteur fixe, sans aucune notion de ce que la découpe traverse. Chaque bande a la même hauteur, qu'elle contienne un paragraphe entier, la moitié d'un tableau, ou le bas d'une image et le haut de la section suivante.

La conséquence est mécanique. Une réponse courte noyée dans une bande de 1024 pixels qui agrège trois sections hétérogènes devient très difficile à retrouver. De ce fait, le signal utile représente quelques mots sur plusieurs centaines. Symétriquement, un tableau coupé au milieu produit deux unités dont aucune ne porte l'information complète. Le système de récupération se voit alors confier une tâche supplémentaire qui n'était pas la sienne : reconstruire, à partir des pixels, les frontières sémantiques du document.

L'observation qui fonde ce travail est que cette reconstruction est inutile pour une page web. Le moteur de mise en page du navigateur a déjà calculé la boîte englobante exacte de chaque élément avant qu'un seul pixel ne soit peint. La segmentation que le RAG visuel dépense de l'effort à reconstruire est disponible, exacte, et gratuite, dans le DOM de la page rendue. Il ne s'agit ni d'un modèle à entraîner, ni d'une heuristique de segmentation visuelle à calibrer, ni d'un détecteur de frontières appris : il s'agit d'une information que le navigateur expose par une simple requête. La figure 1 montre directement ce que cela change sur une page réelle.

Contribution personnelle

L'intégralité du système décrit dans ce rapport, avec la conception du tuilage, la construction du graphe, l'entraînement contrastif du lecteur, le contrôleur budgété, ainsi que le code, les jeux de données synthétiques et les évaluations qui les accompagnent ont été conçues et implémentées par l'auteur dans le cadre de ce projet de master. Les deux seuls emprunts à des travaux tiers, explicitement identifiés comme tels tout au long du rapport, sont l'architecture de graphe et de contrôleur de MAGE-RAG [45], dont ce travail généralise le contrôleur à des frontières inter-documents qu'il ne traite pas, et le protocole de grille de PixelRAG [37], reproduit à l'identique dans un module indépendant pour servir de condition de référence à l'section 3.2. Tout le reste est une contribution originale de ce projet.

Une version antérieure et condensée des chapitres 2 et 3 a été soumise en double aveugle, sous le titre « *DOM-Graph RAG : DOM-Guided Adaptive Tiling and Budgeted Online Evidence Control for Multimodal Web-Page RAG* » [11], à l'atelier *DocInsights 2026*, associé à EMNLP 2026. Le présent rapport en constitue la version longue : il ajoute le contexte

scientifique complet (chapitre 1), le détail d’implémentation et d’ingénierie nécessaire à la reproduction, et l’analyse critique des quatre évaluations du chapitre 3, dont la version courte ne pouvait rapporter que les résultats les plus saillants.

Objectifs et contributions

L’objectif de ce projet de master est de transformer l’observation ci-dessus en un système complet et de mesurer ce qu’elle apporte réellement. Cela suppose de construire quatre composants et de les évaluer séparément puis ensemble. Les contributions de ce travail sont les suivantes.

1. **Un tuilage adaptatif guidé par le DOM** (section 2.3) qui remplace la grille de pixels aveugle de PixelRAG : chaque élément est recadré sur sa propre boîte englobante, les éléments trop petits sont fusionnés avec leurs voisins, ceux trop grands sont subdivisés, et les titres sont systématiquement appariés avec leur contenu. Le point technique le plus délicat est la mesure de la boîte englobante des balises de texte coulant, dont la boîte CSS s’étend sur toute la largeur du conteneur même lorsque le texte s’enroule autour d’un élément flottant. C’est cette étape, formalisée en détail au chapitre 2, qui porte la recherche centrale de ce travail : aligner la grille de *patches* que consomme le modèle vision-langage sur les frontières sémantiques du DOM plutôt que sur une géométrie arbitraire, de sorte que chaque unité visuelle soumise à l’encodeur porte un contenu homogène et complet.
2. **Un graphe de preuves multigranulaire** (section 2.4) organisant les tuiles en deux niveaux (page et élément) et instanciant les cinq relations du graphe de MAGE-RAG [45], augmentées de deux relations propres à notre contexte : `links_to`, qui suit les liens hypertextes pour relier plusieurs pages, et `continues`, qui chaîne les sections d’une page trop longue pour être traitée d’un bloc.
3. **Un lecteur vision-langage entraîné par contraste** (section 2.5) sur des paires (question, tuile) générées synthétiquement avec négatifs difficiles minés, adapté par LoRA sur le décodeur et la tour de vision, via une tête d’encodage de type PromptEOL [17].
4. **Un contrôleur de preuves en ligne budgété** (section 2.6) qui fait évoluer l’état de chaque nœud du graphe selon une politique best-first sous budget, généralisant l’algorithme de contrôle de MAGE-RAG à des frontières qui franchissent les limites de document.

À ces contributions logicielles s’ajoute une contribution expérimentale : **quatre évaluations** (chapitre 3) qui isolent chaque composant puis mesurent le système assemblé, avec une attention particulière portée à la comparabilité des métriques. Nous montrons notamment que le Recall@ k brut n’est pas comparable entre conditions de tuilage, puisqu’un vivier de candidats plus petit le gonfle mécaniquement, et nous utilisons à la place un rang percentile invariant à la taille du vivier.

Résultats principaux

Deux résultats sont nets. Le tuilage guidé par le DOM double presque la proportion de questions dont la réponse survit dans une unité récupérable (51,8 % contre 26,4 % pour la grille de PixelRAG), tout en consommant 2,7 fois moins de pixels par page ; ces deux effets vont dans le même sens, ce qui est contre-intuitif puisque notre tuilage produit cinq fois

plus de tuiles. L’entraînement contrastif du lecteur fait presque doubler le Recall@1 sur un partitionnement par article strictement disjoint ($0,433 \rightarrow 0,785$).

Les composants en aval donnent des résultats plus nuancés, que nous analysons en détail plutôt que de les minimiser. L’avantage de classement du tuilage DOM sur la grille est réel mais modeste, et il s’inverse face à un découpage textuel plat sur le sous-ensemble strictement apparié : nous montrons que ce résultat est cohérent avec le scoreur lexical employé, qui n’exploite aucune information spatiale, et non avec un défaut du tuilage lui-même. Le contrôleur budgété n’établit pas encore de supériorité sur une sélection top- k statique ; nous en diagnostiquons une cause architecturale identifiable, la corrigeons, mesurons le gain, et documentons ce qui reste à faire. Enfin, l’évaluation bout-en-bout montre que le système assemblé échoue par abstention plutôt que par fabulation, ce qui désigne le sous-rappel de preuves comme le verrou restant et pointe vers un correctif précis et déjà instrumenté dans le code.

Organisation du rapport

Le rapport suit une progression en trois chapitres.

Le **chapitre 1** situe brièvement DOM-Graph RAG dans son contexte scientifique avec la formulation du problème de récupération et de ses métriques, le prétraitement des documents visuellement riches, les modèles vision-langage et la famille du RAG visuel, le RAG structuré par graphe, et les techniques d’apprentissage contrastif mobilisées ensuite. Il reste volontairement concis et se clôt sur l’identification précise du verrou et le positionnement de ce travail face à PixelRAG et MAGE-RAG.

Le **chapitre 2** constitue le cœur de ce rapport. Il ouvre sur ce que le navigateur expose réellement de la géométrie d’une page rendue puis suit l’ordre du pipeline : rendu et extraction géométrique, tuilage adaptatif formalisé en détail, construction du graphe, entraînement contrastif du lecteur et contrôleur en ligne. Chaque étape est explicitée par ses choix de conception, sa formalisation mathématique, les cas limites rencontrés sur des pages réelles, et les décisions d’ingénierie qui rendent le pipeline exécutable sur du matériel modeste.

Le **chapitre 3** présente les quatre évaluations, leur analyse critique, les coûts de calcul observés, et une discussion structurée des limites du travail et des directions qu’elles ouvrent.

Un chapitre de conclusion synthétise l’apport et les enseignements du projet, et les annexes rassemblent les hyperparamètres, l’organisation du dépôt, les invites utilisées et les protocoles de reproduction.

Chapitre 1

État de l'art

Ce chapitre situe DOM-Graph RAG dans son contexte scientifique et se limite à ce qui est strictement nécessaire pour situer les choix de conception : le problème du découpage documentaire et ses métriques (section 1.1), le prétraitement des documents visuellement riches (section 1.2), la famille du RAG visuel et son représentant le plus proche, PixelRAG (section 1.3), le RAG structuré par graphe et son ancêtre direct, MAGE-RAG (section 1.4), et l'apprentissage contrastif mobilisé par le lecteur (section 1.5). Il se clôt sur le positionnement précis de ce travail (section 1.6).

1.1 Génération augmentée par récupération et découpage

La figure 1.1 donne les trois opérateurs qu'enchaîne un système RAG. Le découpeur $C : \mathcal{D} \mapsto \mathcal{U}$ transforme une collection de documents en unités récupérables. Le récupérateur $R : q \mapsto \mathcal{U}_k \subseteq \mathcal{U}$ sélectionne les k unités les plus pertinentes pour une question q selon un score $s(q, u)$. Le générateur $G : (q, \mathcal{U}_k) \mapsto a$ répond à partir de la question et de ces unités.

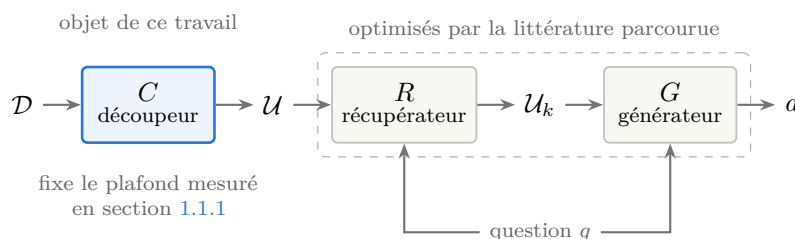


Figure 1.1 – Les trois opérateurs d'un système RAG. La composition impose un ordre de dépendance, car R ne peut sélectionner que des unités produites par C , et G ne peut répondre qu'à partir de ce que R transmet.

Lewis et al. [23] ont formalisé cette architecture et montré qu'affiner conjointement l'encodeur de requête et le générateur améliore les tâches à forte intensité de connaissances ; Izacard and Grave [16] ont montré qu'encoder chaque passage séparément avant de les fusionner dans le décodeur permet d'en traiter un grand nombre sans coût quadratique. Gao et al. [13] recensent ces variantes sans qu'aucune ne s'impose.

Le score $s(q, u)$ oppose deux familles. Les approches lexicales (TF-IDF, BM25 [31]) ne demandent aucun entraînement et restent compétitives : le *benchmark* BEIR [32] établit qu'en transfert zéro-coup BM25 n'est pas systématiquement dépassé par les récupérateurs denses, et la section 3.3 retrouve ce constat. Leur limite est le décalage de vocabulaire. Les approches denses projettent question et unité dans un espace vectoriel commun appris par contraste [12, 30] ; Karpukhin et al. [19] en tirent la recette de référence pour la récupération de passages, et ColBERT [20] en conserve un vecteur par token plutôt qu'un vecteur unique, une idée qui réapparaît dans le RAG visuel (section 1.3).

Cette littérature suppose C donné, alors qu'il fixe un plafond que ni R ni G ne peuvent franchir. Si la réponse à q est répartie sur deux unités distinctes, ou noyée dans une unité majoritairement hors sujet, aucun score ne peut compenser. Les stratégies de découpage textuel, fenêtres de taille fixe, découpage guidé par la structure du document ou découpage

sémantique aux points de rupture de similarité entre plongements [27], sont recensées par Gao et al. [13] sans que leur effet sur ce plafond soit mesuré. Toutes opèrent sur du texte déjà extrait, où les frontières de mise en page ont été détruites, alors qu’un moteur de rendu les calcule sans coût additionnel sur le web (section 2.1). Toutes mesurent aussi la qualité du classement en supposant l’attachement de la réponse acquis, ce que ne fait ni la section 1.1.1 ni la section 3.2.2.

1.1.1 Le biais de taille de vivier

Comparer deux stratégies de découpage produisant des viviers de candidats de tailles différentes pose un piège méthodologique peu discuté : sur une même page en 5 bandes contre 25 tuiles, un classement aléatoire place la bonne unité au premier rang avec une probabilité de 20 % dans le premier cas et de 4 % dans le second, de sorte que comparer des Recall@1 bruts revient en partie à comparer des tailles de vivier. Le rang percentile

$$\text{pct}(u^+) = 1 - \frac{\text{rang}(u^+) - 1}{n_{\text{items}} - 1}, \quad (1.1)$$

vaut 0,5 en espérance sous classement aléatoire quelle que soit la taille du vivier, et constitue la métrique de comparaison entre conditions de tuilage employée en section 3.2, aux côtés du taux d’attachement qui représente la proportion de questions dont la réponse survit au découpage, absente des protocoles usuels alors qu’elle détermine le plafond que ni R ni G ne peuvent franchir.

1.2 Documents visuellement riches et prétraitement

Un document est dit visuellement riche lorsque sa mise en page porte de l’information sémantique, comme un tableau, une facture ou une page à encadrés. Pour un tel document, la chaîne de prétraitement classique enchaîne trois étapes à perte, détaillées par la figure 1.2. Le résultat cumulé est qu’une question du type « quelle est la valeur de la colonne revenus pour la ligne 2024 » devient difficile à satisfaire, non parce que le récupérateur est mauvais, mais parce que la correspondance ligne-colonne a été détruite en amont.

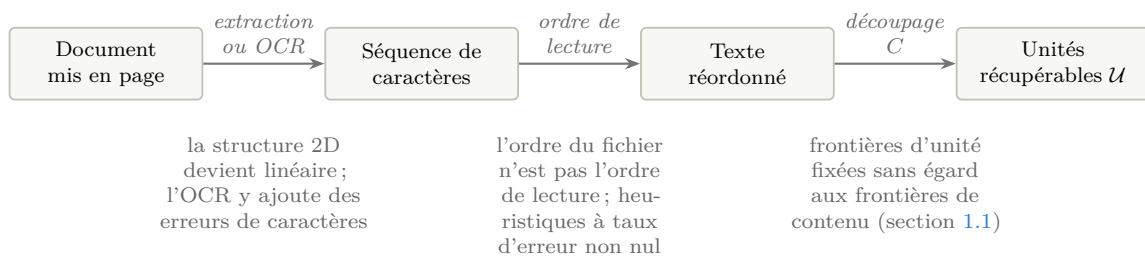


Figure 1.2 – Les trois étapes à perte de la chaîne de prétraitement classique et l’information détruite à chacune. Les modèles sensibles à la mise en page réinjectent la géométrie en aval de la première étape ; les modèles sans OCR suppriment les deux premières.

Une première réponse injecte la géométrie dans la représentation. LayoutLM [40] ajoute aux plongements de tokens des positions 2D dérivées de la boîte englobante de chaque mot ; LayoutLMv2 [39] et LayoutLMv3 [15] l’intègrent au pré-entraînement (92,1 de F_1 sur FUNSD, 83,4 d’ANLS sur DocVQA) ; MarkupLM [24] transpose l’idée au web en encodant le chemin XPath de chaque token. Ces modèles reçoivent tous des mots et boîtes déjà produits par un OCR en amont, et la structure y est apprise, donc coûteuse à ré-entraîner sur un type de document non couvert ; surtout, elle sert la représentation d’une unité déjà découpée,

non la décision de découpage : MarkupLM sait que deux tokens partagent un ancêtre DOM, il ne décide pas où couper la page.

Une seconde réponse supprime l’OCR : Donut [21] produit du texte structuré directement depuis l’image d’un document, et Pix2Struct [22] porte l’idée au web en pré-entraînant sur 80 millions de captures d’écran, avec pour cible un HTML simplifié dérivé du DOM rendu. Cela permet ainsi d’établir que le DOM porte la structure dont un modèle visuel a besoin, mais il l’exploite comme cible de supervision apprise, quand ce travail l’emploie comme décision de découpage à l’inférence, sans apprentissage.

1.3 Modèles vision-langage et RAG visuel

Les modèles vision-langage contemporains combinent une tour de vision de type *Vision Transformer* [8], qui découpe l’image en *patches* traités par auto-attention [35], et un modèle de langue décodeur consommant une séquence mixte de tokens visuels et textuels, sur la base des représentations image-texte alignées par contraste [28]. Qwen2-VL [36], employé au chapitre 2, apporte le support de la résolution dynamique : le nombre de tokens visuels y est proportionnel à l’aire réelle de l’image, une propriété dont la section 2.3.4 tire les conséquences pour ce travail.

Si un tel modèle produit un plongement d’image aligné sur le langage, on peut indexer des images de pages entières et récupérer dessus : *Document Screenshot Embedding* [25] le fait sans extraction de contenu préalable, et ColPali [10] porte le mécanisme d’interaction tardive de ColBERT aux *patches* visuels.

1.3.1 PixelRAG et le découpage aveugle

Le travail le plus directement relié est PixelRAG [37], qui applique cette approche à l’échelle du web : les pages sont rendues, découpées en bandes et indexées avec un encodeur vision-langage, et les auteurs rapportent égalier ou dépasser la récupération textuelle sur plusieurs bancs d’essai, avec une réduction du coût en tokens allant jusqu’à un facteur trois par sous-échantillonnage de la résolution. C’est son découpage qui nous intéresse : une grille de hauteur fixe, la page rendue tranchée en bandes de 875×1024 pixels indépendamment de leur contenu. La décision est simple et sans dépendance au type de document, mais son défaut est celui de la fenêtre de taille fixe qui crée une bande coupant là où elle tombe, au milieu d’un tableau comme d’un paragraphe. C’est cette condition qui est réimplémentée (section 2.8.2) et mesurée (section 3.2).

1.4 RAG structuré par graphe

Traiter $\mathcal{U}$ comme un sac plat empêche de récupérer une unité peu pertinente lexicalement mais indispensable contextuellement, et interdit le raisonnement multi-sauts. Deux issues coexistent : itérer la récupération ou structurer les preuves en amont. La première voie est celle de ReAct [42], IRCoT [33], FLARE [18] et Self-RAG [1], qui entrelacent raisonnement et appels de récupération, au prix d’appels au modèle en série. Dans la seconde, Edge et al. [9] construisent un graphe de connaissances sur des entités extraites par un modèle de langue, ce qui répond à des questions globales sur un corpus qu’aucune récupération locale ne satisferait ; le graphe porte sur des entités, non sur la structure documentaire.

1.4.1 MAGE-RAG

Zuo et al. [45] construisent au contraire un graphe de la structure du document, et c’est l’ancêtre direct de l’architecture proposée ici : des nœuds de niveau page et de niveau élément, reliés par cinq relations (`contains`, `reading_order`, `layout_adjacency`, `section_hierarchy`, `semantic_neighbor`), que parcourt à la requête un contrôleur budgété décidant quels nœuds le modèle lecteur ouvre réellement. L’section 2.6 en généralise l’algorithme. La différence tient à la provenance du graphe : MAGE-RAG le construit hors ligne, à partir d’un document déjà segmenté et d’un ordre de lecture déjà déterminé, quand celui construit ici l’est depuis le DOM d’une page rendue, qui fournit sans coût additionnel frontières exactes et types d’éléments (section 2.1). Deux relations que le contexte web rend nécessaires s’y ajoutent, `links_to` et `continues` (tableau 2.2). MAGE-RAG est évalué sur LongDocURL et MMLongBenchDoc, deux jeux de données constitués de PDF déjà analysés ; ces chiffres sont rapportés au chapitre 3 comme référence de l’architecture, non comme comparaison numérique, faute d’un DOM à exploiter sur un PDF.

1.5 Apprentissage contrastif et adaptation efficace

1.5.1 InfoNCE et la qualité des négatifs

Entraîner un récupérateur revient à apprendre une similarité $s(q, u)$ qui place la bonne unité plus près de la question que les mauvaises. L’objectif standard, InfoNCE [34], largement repris en vision [3], s’écrit pour une question de plongement $\mathbf{z}_q$, une unité positive u^+ et des négatifs $\{u_1^-, \dots, u_K^-\}$:

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(\text{sim}(\mathbf{z}_q, \mathbf{z}_{u^+})/\tau)}{\exp(\text{sim}(\mathbf{z}_q, \mathbf{z}_{u^+})/\tau) + \sum_{i=1}^K \exp(\text{sim}(\mathbf{z}_q, \mathbf{z}_{u_i^-})/\tau)}, \quad (1.2)$$

où τ est une température qui concentre le gradient sur les négatifs proches du positif à mesure qu’elle diminue. Tout dépend des négatifs : les *in-batch negatives* [19] réutilisent gratuitement les positifs des autres exemples du lot, tandis que les négatifs difficiles sont minés délibérément. La section 2.5 reprend cette équation, l’étend au lot entier, et l’analyse plus en détail. La génération synthétique de paires question-passage, établie par InPars [2] et Promptagator [4], est transposée ici à des tuiles d’image.

1.5.2 Adaptation efficace d’un modèle génératif en encodeur

Affiner intégralement un modèle de plusieurs milliards de paramètres est hors de portée d’une infrastructure académique modeste. LoRA [14] gèle les poids pré-entraînés et n’apprend, pour chaque matrice ciblée, qu’une mise à jour de rang faible ; QLoRA [7] y ajoute la quantification du modèle gelé. Dans le cas multimodal, LoRA peut cibler le décodeur seul ou aussi la tour de vision puisque la tâche traitée au section 2.5.4 exige les deux. Un second obstacle est que ces modèles sont génératifs et n’exposent aucun vecteur de représentation ; PromptEOL [17] fait suivre l’entrée d’une amorce à compléter et prend l’état caché du dernier token comme plongement, un procédé appliqué à l’identique à une question textuelle et à une image de tuile au section 2.5.3.

1.6 Synthèse et positionnement

Trois constats délimitent une lacune précise, que le tableau 1.1 résume.

Tableau 1.1 – Origine des frontières de l’unité récupérable chez les travaux les plus proches. Elles sont soit arbitraires, fenêtre ou grille, soit héritées d’un découpage effectué ailleurs, format du fichier source ou corpus déjà segmenté en amont.

Travail	Origine des frontières
<i>Récupération textuelle</i>	
DPR [19]	fenêtre fixe
GraphRAG [9]	fenêtre fixe
<i>Modèles sensibles à la mise en page</i>	
LayoutLM [40]	corpus segmenté en amont
MarkupLM [24]	corpus segmenté en amont
<i>RAG visuel, lecture des pixels</i>	
DSE [25]	format du fichier source, la page
ColPali [10]	format du fichier source, la page
PixelRAG [37]	grille de hauteur fixe
<i>RAG par graphe de structure</i>	
MAGE-RAG [45]	corpus segmenté en amont
Ce travail, pixels et graphe	boîtes englobantes du DOM rendu

Premièrement, le RAG visuel démontre que lire les pixels préserve une information que l’extraction textuelle détruit, mais ne traite pas le découpage : soit il l’évite en travaillant à l’échelle de la page, soit il le tranche à l’aveugle, comme PixelRAG. Deuxièmement, les modèles sensibles à la mise en page exploitent la structure, mais en l’apprenant à grands frais et pour représenter des unités déjà découpées. Enfin, MAGE-RAG démontre qu’un contrôleur de graphe budgété surpasse une sélection top- k statique, mais suppose les éléments du document déjà constitués en amont de son propre pipeline, c’est-à-dire l’artefact que PixelRAG jette au rendu et que ce travail restitue.

La thèse défendue est que, pour une page web sémantiquement balisée, la segmentation dont le RAG visuel a besoin est déjà disponible et exacte dans le DOM de la page rendue, et qu’il est donc inutile de la reconstruire depuis les pixels ou de la contourner par une grille. Le chapitre 2 formalise et met à l’épreuve cette thèse, en commençant précisément là où PixelRAG s’arrête : par ce qu’un navigateur expose de la géométrie d’une page rendue.

Chapitre 2

Méthodologie

Ce chapitre décrit le système construit pour mettre à l'épreuve la thèse formulée au chapitre précédent, et en détaille la formalisation mathématique. Il commence par établir ce qu'un navigateur expose réellement de la géométrie d'une page rendue (section 2.1) représentant le préalable technique dont dépendent tous les choix qui suivent, puis suit l'ordre de la chaîne de traitement : une page est rendue une fois par un navigateur sans interface graphique et sa géométrie est mesurée (section 2.2) ; cette géométrie pilote un tuilage adaptatif, dont la section 2.3 formalise les règles et le modèle de coût (section 2.3) ; les tuiles sont organisées en un graphe à deux niveaux, éventuellement étendu à plusieurs pages liées (section 2.4) ; un lecteur vision-langage est entraîné par contraste, hors ligne, à évaluer la pertinence d'une paire (question, tuile) (section 2.5) ; enfin, à la requête, un contrôleur en ligne parcourt le graphe sous budget pour décider quels nœuds le lecteur ouvre réellement (section 2.6). La section 2.7 détaille les choix d'ingénierie qui rendent l'ensemble exécutable, et la section 2.8 la construction des jeux de données évalués au chapitre 3.

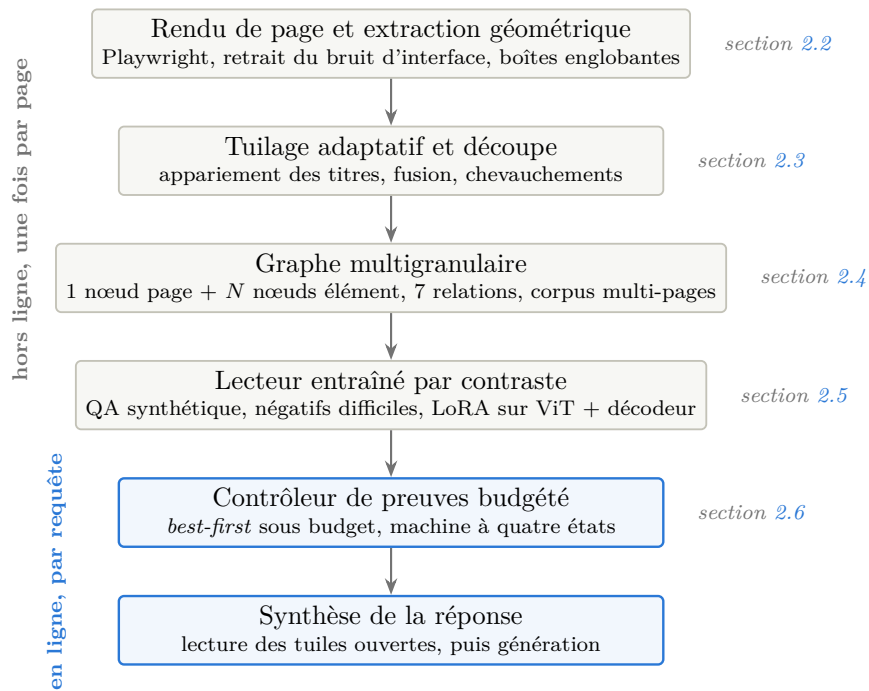


Figure 2.1 – Vue d'ensemble de la chaîne DOM-Graph RAG

2.1 Préliminaires géométriques : le DOM et le moteur de rendu

L'hypothèse centrale de ce travail est que la segmentation nécessaire au RAG visuel est déjà présente dans le DOM d'une page rendue. Cette section établit quelle information le navigateur expose, où se situent les pièges, et pose la notation reprise par le reste du chapitre.

2.1.1 Du balisage à la mise en page, et le modèle de boîtes

Le traitement d'une page par un navigateur enchaîne quatre étapes. L'analyse du HTML produit le DOM, un arbre $T = (V, E)$ dont chaque nœud $v \in V$ correspond à un élément ou à un nœud texte. La cascade attribue ensuite à chaque élément ses valeurs de propriétés calculées. La mise en forme (*reflow*) détermine la position et les dimensions exactes de chaque boîte dans un repère absolu. La peinture produit enfin les pixels.

La mise en forme se termine intégralement avant que le premier pixel ne soit peint, si bien que la géométrie de tout élément est déjà connue au moment de la peinture. Un système qui recadre des régions dans l'image peinte travaille donc en aval d'une source de vérité géométrique qu'il n'a jamais consultée.

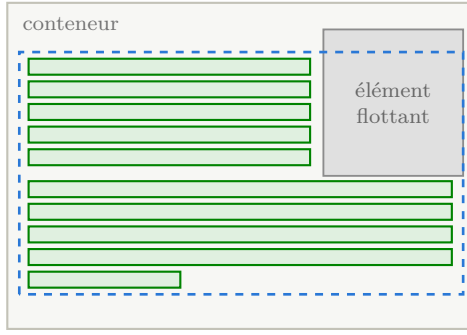
La fonction $\mathcal{B} : V \rightarrow \mathbb{R}_{\geq 0}^4$ associe à un élément rendu sa boîte $\mathcal{B}(v) = (x_v, y_v, w_v, h_v)$ dans le repère absolu du document, où (x_v, y_v) est le coin supérieur gauche et (w_v, h_v) la largeur et la hauteur. C'est elle qu'exploite le tuilage de la section 2.3, aucune partie de la chaîne ne redérivant une géométrie que le navigateur a déjà calculée.

Une boîte est faite de quatre régions concentriques, contenu, *padding*, bordure et marge. Les API de géométrie rapportent par défaut celle qui correspond visuellement à l'élément, et $\mathcal{B}$ désigne implicitement cette région.

Le type d'affichage détermine le comportement de $\mathcal{B}$. Un élément de type **bloc** (`<p>`, `<div>`, `<h1>`–`<h6>`, `<ul>`) a par défaut `width: auto`, donc $w_v = w_{\text{parent}(v)}$ quel que soit son contenu. Un titre de trois mots dans une colonne de 1 200 px a ainsi une boîte de 1 200 px de large. Un élément de type **en ligne** (`<span>`, `<a>`, `<em>`) ne forme pas une boîte unique mais autant de rectangles que de fragments de ligne. Un `<table>`, enfin, s'ajuste à son contenu au lieu de remplir son conteneur. La difficulté décrite ci-dessous ne frappe donc que les blocs de texte.

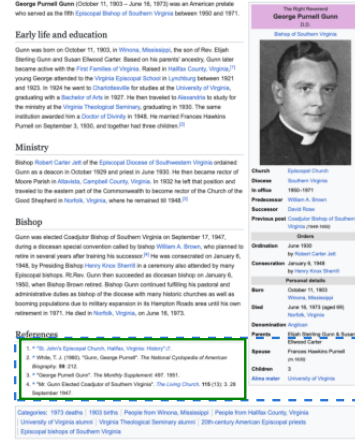
2.1.2 Éléments flottants : l'écart entre boîte de bloc et lignes réelles

Le cas déterminant pour la mesure des tuiles est celui des éléments flottants. Un élément déclaré `float: right` est retiré du flux normal et poussé sur le bord droit de son conteneur, et le contenu en ligne des blocs qui le suivent s'enroule autour de lui. L'effet est asymétrique. Pour un bloc v qui suit un flottant, $\mathcal{B}(v)$ n'est pas réduite et continue de s'étendre sur $w_{\text{parent}(v)}$ en passant sous le flottant, alors que seules les lignes de texte à l'intérieur de v sont raccourcies. La figure 2.2 met les deux géométries en regard.



--- getBoundingClientRect
 ■■■ getClientRects, nœuds texte

(a) Les deux géométries d'un même bloc



(b) La liste de références d'une page réelle

Figure 2.2 – (a) Un élément flottant à droite raccourcit les lignes des blocs qui le suivent sans réduire leur boîte de bloc, laquelle continue de passer sous lui. (b) La boîte de bloc de la liste de références, en tirets, s'étend sous l'infoboîte, alors que l'union de ses rectangles de lignes s'arrête à son bord

La correction employée en section 2.2.4 se formalise ainsi. Soit v un élément de texte coulant et $N_1, \dots, N_k$ ses nœuds texte descendants. `getClientRects`, appliqué à un objet `Range` construit sur chaque N_i , retourne les rectangles de ligne effectivement rendus $\{r_{i,1}, \dots, r_{i,m_i}\}$. Chaque r est un quadruplet $(x^{(0)}, y^{(0)}, x^{(1)}, y^{(1)})$ de coordonnées de coins. La boîte serrée est l'enveloppe rectangulaire de leur union.

$$\mathcal{B}_{\text{serré}}(v) = \left[\min_{i,j} x_{i,j}^{(0)}, \min_{i,j} y_{i,j}^{(0)}, \max_{i,j} x_{i,j}^{(1)} - \min_{i,j} x_{i,j}^{(0)}, \max_{i,j} y_{i,j}^{(1)} - \min_{i,j} y_{i,j}^{(0)} \right]. \quad (2.1)$$

Le `Range` doit être construit sur les nœuds texte et non sur l'élément lui-même. Un `Range` englobant tout v retournerait en plus le rectangle de sa boîte pleine largeur, ce qui ramènerait l'union à $\mathcal{B}(v)$. Un nœud texte n'a par construction aucune boîte propre, et ses rectangles ne peuvent donc être que des lignes réellement rendues.

En l'absence de flottant, $\mathcal{B}_{\text{serré}}(v) = \mathcal{B}(v)$, les lignes remplissant la largeur du bloc. En présence d'un flottant qui intersecte les rangées de v , la boîte serrée est strictement plus étroite, et l'écart d'aire

$$\Delta A(v) = \text{aire}(\mathcal{B}(v)) - \text{aire}(\mathcal{B}_{\text{serré}}(v)) \geq 0 \quad (2.2)$$

mesure le fond blanc qu'un recadrage sur $\mathcal{B}(v)$ soumettrait en pure perte à l'encodeur visuel. C'est ce terme, sommé sur toutes les balises de texte coulant d'une page, que la mesure serrée de la section 2.2.4 élimine, et dont la section 3.2.3 rapporte l'effet agrégé sur le pixels par page.

Pour un `<table>`, un `<figure>` ou un `<img>`, en revanche, bordure, fond et marge intérieure font partie du contenu visuel. La boîte brute $\mathcal{B}(v)$ y reste la mesure correcte, et $\Delta A(v) = 0$ par convention.

2.1.3 Les deux API de géométrie et le repère absolu

Le navigateur expose deux méthodes de mesure. `getBoundingClientRect` retourne $\mathcal{B}(v)$, et `getClientRects` la liste des rectangles effectivement occupés, base de $\mathcal{B}_{\text{serré}}$ ci-dessus. Toutes deux sont relatives au *viewport* et non au document, alors que le recadrage d'une capture

pleine page exige le repère absolu. Le décalage de défilement courant (s_x, s_y) leur est donc ajouté.

$$\mathcal{B}_{\text{abs}}(v) = \mathcal{B}_{\text{viewport}}(v) + (s_x, s_y, 0, 0). \quad (2.3)$$

Cette conversion est appliquée systématiquement à l'extraction (section 2.2). La profondeur d'un nœud dans T fournit par ailleurs un ordre de traitement commode pour résoudre l'imbrication (section 2.3.1), un parent étant toujours à une profondeur strictement inférieure à celle de ses descendants.

2.1.4 Ordre du DOM et ordre visuel

L'ordre des nœuds dans T est celui dans lequel l'auteur a écrit le balisage, et non l'ordre de lecture visuel, le positionnement CSS pouvant faire diverger les deux arbitrairement. Le déduire du parcours de T produirait, sur une mise en page non triviale, une séquence qui ne correspond pas à ce qu'un lecteur humain suit. La relation d'ordre de lecture (section 2.4.1) est donc calculée par une heuristique spatiale sur les coordonnées $\mathcal{B}_{\text{abs}}$ plutôt que par un parcours de T . Le DOM fournit $\mathcal{B}$ et les types d'éléments, pas un ordre de lecture total.

2.2 Rendu de page et extraction géométrique

2.2.1 Rendu sans interface graphique

La page est chargée sans interface graphique, via Playwright, avec une largeur de fenêtre d'affichage de 2048 px et un facteur d'échelle de périphérique de 1. La hauteur de la fenêtre, fixée à 1000 px, est sans effet, la capture pleine page produisant une image de la hauteur totale du document rendu, soit plusieurs dizaines de milliers de pixels sur les articles les plus longs (section 2.3.5).

La largeur de 2048 px égale la hauteur maximale de tuile (section 2.3.3), de sorte que la plus grande unité produite est un carré de 2048 pixels de côté et qu'un seul paramètre borne l'aire maximale soumise à l'encodeur. Elle n'améliore en revanche pas la résolution du texte, la taille en pixels d'un caractère ne dépendant pas de la largeur de fenêtre à facteur d'échelle constant. Élargir le rendu ajoute de la marge, non de la définition.

Cette marge n'est pas neutre pour le chapitre 3. La colonne de contenu occupe environ 40 % de la largeur de rendu (figure 2.3), si bien qu'une bande de grille est majoritairement du fond blanc là où une tuile mesurée serrée (section 2.2.4) ne l'est pas. Le rapport de budget de pixels de la section 3.2.3 est mesuré à cette largeur et décroît avec elle.

Ordre des opérations. Le chargement attend la condition `networkidle`, et l'extraction géométrique est évaluée après la capture pleine page, la condition d'attente étant réévaluée entre les deux. La capture fait en effet défiler le document et déclenche le chargement différé des images, qui modifie la hauteur des blocs qui les contiennent. Une mesure prise avant ce défilement décrirait une mise en page antérieure au chargement, et les recadrages seraient décalés d'autant sans qu'aucune métrique ne le signale.

2.2.2 Retrait du bruit d'interface

Avant la capture, un ensemble de sélecteurs CSS retire du document le bruit d'interface, c'est-à-dire les barres de navigation, pieds de page, panneaux latéraux du logiciel de wiki, bandeaux d'avertissement, liens d'édition de section et boîtes de navigation. Ces éléments ne portent aucune information sur le sujet de la page et produiraient des tuiles coûteuses en pixels et systématiquement non pertinentes.

La liste combine des sélecteurs génériques (`nav`, `footer`, `header`, `script`, `style`) et des sélecteurs propres à MediaWiki (`#mw-navigation`, `.vector-header`, `.navbox`, `.mw-editsection`, `.noprint`). Cette seconde catégorie est une dépendance explicite au type de site et constitue la principale limite de portée des résultats quantitatifs (section A.5). Elle porte sur le nettoyage et non sur la logique de tuilage.

2.2.3 Extraction des boîtes englobantes

L'extraction est un unique appel de fonction JavaScript évalué dans le contexte de la page. Pour un ensemble configurable de balises (`h1` à `h4`, `p`, `table`, `figure`, `img`, `ul`, `ol`, `blockquote`, `pre`), le script relève, pour chaque élément correspondant, sa position absolue au sens de l'équation (2.3), ses dimensions, un aperçu de son texte, sa profondeur dans l'arbre DOM et, le cas échéant, les liens hypertextes qu'il contient.

Extraction sélective des liens. Les liens ne sont relevés que pour les éléments `p`. Une page encyclopédique porte quelques dizaines de liens dans son texte et plusieurs centaines dans ses tableaux, boîtes de navigation et listes de références, dont le suivi ferait croître le coût du parcours de corpus (section 2.4.6) sans apporter de contenu pertinent. Les liens conservés sont résolus en URL absolue via `document.baseURI`, les ancres internes et les schémas non HTTP étant écartés.

2.2.4 Mesure serrée des balises de texte coulant

Recadrer sur $\mathcal{B}$ plutôt que sur $\mathcal{B}_{\text{serré}}$ produit deux défauts mesurables, l'un et l'autre conséquences de l'asymétrie formalisée en section 2.1.2.

Le premier est un double comptage. Recadrer sur $\mathcal{B}(v)$ capture les pixels de l'élément flottant voisin en plus du contenu visé, et ces pixels apparaissent alors dans deux unités de récupération, celle du bloc et celle de l'infoboîte. Le budget est payé deux fois, et les scores de pertinence de deux tuiles sont alimentés par le même contenu.

Le second est un coût. Une tuile de texte recadrée sur $\mathcal{B}(v)$ consomme la même surface qu'une bande de grille quel que soit son contenu réel, soit 2 058 000 pixels en moyenne contre 157 170 pour une tuile recadrée sur $\mathcal{B}_{\text{serré}}(v)$ (section 3.2.3). Le cas dominant est la liste de références en fin d'article, rendue en `<ol>`, dont la boîte traverse toute la largeur du conteneur alors que ses lignes occupent une colonne étroite (figure 2.2b).

Le tuilage recadre donc sur $\mathcal{B}_{\text{serré}}(v)$ pour les balises dont la boîte de bloc est structurellement plus large que leur contenu (`h1` à `h4`, `p`, `blockquote`, `ul`, `ol`), désignées balises de texte coulant et configurables. Un `TreeWalker` parcourt les nœuds texte non vides de v , un `Range` est construit sur le contenu de chacun, et l'équation (2.1) est évaluée sur l'union des rectangles retournés par `getClientRects`. La granularité du `Range` est imposée par la contrainte établie en section 2.1.2. Les autres balises conservent $\mathcal{B}(v)$, avec $\Delta A(v) = 0$.

2.3 Tuilage adaptatif guidé par le DOM

Le tuilage instancie pour le cas web le découpeur $C : \mathcal{D} \mapsto \mathcal{U}$ de la section 1.1. La fonction C_{DOM} définie ci-dessous, construite à partir de $\mathcal{B}$ et $\mathcal{B}_{\text{serré}}$ (section 2.1), remplace la grille de hauteur fixe de PixelRAG. Elle transforme une liste d'éléments géolocalisés E et une capture pleine page I en une liste de tuiles $T = C_{\text{DOM}}(E, I)$, chacune formée d'un recadrage d'image accompagné de ses coordonnées, de la balise DOM d'origine, d'un aperçu textuel, des identifiants des éléments sources et des liens agrégés.

Le nombre de tuiles retenues par unité de page est borné à 25. Cette borne est héritée du coût de génération des paires question-réponse (section 2.8) et restreint la portée de

l'évaluation A (section A.5). Trois traitements préparatoires précèdent le tuilage proprement dit.

2.3.1 Élagage des éléments imbriqués

Les balises retenues s'imbriquent, un `<img>` dans un `<figure>`, un `<p>` dans une cellule de `<table>`, une liste `<ul>` dans un `<blockquote>`. Sans traitement, chaque niveau produirait sa propre tuile et le contenu de l'enfant apparaîtrait deux fois.

L'élagage traite les éléments par profondeur DOM croissante et ne compare chaque candidat qu'aux éléments déjà retenus. Le candidat est écarté si sa boîte est entièrement contenue dans celle d'un élément retenu, à une tolérance d'un pixel près qui absorbe les arrondis de rendu. L'ordre par profondeur croissante garantit qu'un parent est examiné avant ses descendants, donc qu'un enfant n'est jamais comparé à un autre enfant lui-même susceptible d'être élagué.

2.3.2 Résolution des chevauchements partiels

L'élagage ne traite que les inclusions complètes. Deux éléments indépendants peuvent se chevaucher sans que l'un contienne l'autre, même après la mesure serrée de la section 2.2.4. Une infoboîte flottante et le bloc qui partage sa rangée s'intersectent ainsi sur quelques lignes, là où le texte reprend la pleine largeur sous l'élément flottant. Les deux sont conservés, mais les tuiler séparément dupliquerait la région commune. Ils sont donc réunis en un seul recadrage couvrant leur union. Le critère de regroupement est le rapport de l'aire d'intersection à l'aire du plus petit des deux éléments,

$$\rho(a, b) = \frac{\text{aire}(B_a \cap B_b)}{\min(\text{aire}(B_a), \text{aire}(B_b))}, \quad (2.4)$$

deux éléments étant regroupés dès que $\rho(a, b) \geq 0,15$. Le seuil strictement positif absorbe les recouvrements de quelques pixels dus aux marges et aux bordures.

Le regroupement s'effectue par composantes connexes sur cette relation, au moyen d'une structure *union-find*. La transitivité est nécessaire. Si a chevauche b et b chevauche c sans que a et c ne se chevauchent, les trois doivent finir dans le même groupe, faute de quoi la tuile de $\{a, b\}$ et celle de $\{b, c\}$ partageraient les pixels de b .

Les titres sont exclus du regroupement, la règle d'appariement de la section 2.3.3 exigeant qu'ils restent des éléments isolés en attente.

2.3.3 Les trois règles de tuilage

Les groupes sont triés de haut en bas puis de gauche à droite et traités séquentiellement selon trois règles.

Appariement des titres. Un titre n'est pas une tuile autonome. Celle d'un `<h2>` seul ne contiendrait que quelques mots sur un fond blanc, coûteuse en unité de récupération et sémantiquement vide, puisqu'un titre sans son contenu ne répond à aucune question. Un titre rencontré est donc mis en attente, et le groupe suivant est fusionné avec lui dans une tuile unique dont la balise composée (`h2+p`) enregistre la fusion, réutilisée en section 2.4.3. Deux cas font exception et émettent un titre seul, deux titres consécutifs et un titre en fin d'unité de page.

Algorithme 1 — Tuilage adaptatif guidé par le DOM

Entrée: éléments E avec boîtes englobantes, capture pleine page I , hauteurs $h_{\max}$ et $h_{\min}$, seuil $\rho_{\min}$

Sortie: liste de tuiles T

```
1  $E \leftarrow \text{ÉLAGUERIMBRIQUÉS}(E)$  // profondeur DOM croissante
2  $G \leftarrow \text{GROUPERCHEVAUUREMENTS}(E, \rho_{\min})$  // union-find
3 trier  $G$  par  $(y, x)$  du premier membre
4  $T \leftarrow \emptyset$ ;  $\text{tampon} \leftarrow \emptyset$ ;  $\text{titre} \leftarrow \perp$ 
5 pour chaque  $g \in G$  faire
6   si  $|g| = 1$  et  $g_1$  est un titre alors
7      $\text{VIDERTAMPON}()$ ;  $\text{VIDERTITRE}()$ 
8      $\text{titre} \leftarrow g_1$  continuer
9   si  $\text{titre} \neq \perp$  alors
10     $T \leftarrow T \cup \text{RECADRERDÉCOUPER}(\{\text{titre}\} \cup g)$ 
11     $\text{titre} \leftarrow \perp$ ; continuer
12   si  $|g| > 1$  alors
13      $\text{VIDERTAMPON}()$ 
14      $T \leftarrow T \cup \text{RECADRERDÉCOUPER}(g)$  continuer
15   si  $\text{hauteur}(g_1) < h_{\min}$  alors
16      $\text{tampon} \leftarrow \text{tampon} \cup \{g_1\}$ 
17     si  $\text{étendue}(\text{tampon}) \geq h_{\min}$  alors
18        $\text{VIDERTAMPON}()$ 
19   sinon
20      $\text{VIDERTAMPON}()$ 
21      $T \leftarrow T \cup \text{RECADRERDÉCOUPER}(\{g_1\})$ 
22  $\text{VIDERTAMPON}()$ ;  $\text{VIDERTITRE}()$ 
23 return  $T$ 
```

Fusion des éléments courts. Un élément non titre dont la hauteur est inférieure à 80 px est mis en tampon. Le tampon s’accumule jusqu’à ce que l’étendue verticale de son union franchisse ce minimum, puis est vidé en une tuile unique de balise `merged`. Sans cette règle, une succession de courtes lignes de catégories en bas de page produirait une avalanche de micro-tuiles quasi vides, consommant chacune une entrée d’index et un appel de génération. La tuile 7 de la figure 2.3 en donne le résultat. Un groupe issu d’un chevauchement (section 2.3.2) échappe à cette règle et n’est jamais mis en tampon, la non-duplication de pixels primant sur l’heuristique de fusion.

Découpe des éléments démesurés. Un élément dont la hauteur dépasse 2048 px, un très long tableau ou une liste de références étendue, est tranché en $\lceil h/h_{\max} \rceil$ sous-tuiles de hauteur égale. C’est le seul endroit où le tuilage recourt à une découpe indifférente au contenu, à l’intérieur d’un unique élément DOM devenu trop grand pour l’encodeur. Cette découpe est une version strictement plus étroite du mode de défaillance de la grille aveugle, signalée comme telle en section A.5.

L’algorithme 1 donne la procédure complète.

$\text{RECADRERDÉCOUPER}(S)$ recadre I sur l’union des boîtes de S , puis lui applique la découpe ci-dessus si la hauteur obtenue dépasse $h_{\max}$. VIDERTAMPON et VIDERTITRE émettent respectivement une tuile fusionnée pour le tampon accumulé et, dans les deux cas d’exception

ci-dessus, une tuile pour un titre resté sans contenu suivant.

Complexité et comptages. L'élagage compare chaque candidat aux éléments déjà retenus et le regroupement examine toutes les paires, ce qui place les deux étapes en $O(n^2)$ dans le pire cas, où n est le nombre d'éléments extraits. La page la plus dense du corpus en compte 278 avant élagage. La chaîne des comptages médians par unité de page s'établit à 9 éléments extraits, 7 après élagage, 7 groupes, pour 6 tuiles. Ces deux étapes restent négligeables devant le rendu de page et, à plus forte raison, devant les appels au modèle vision-langage (section A.4). Une structure d'indexation spatiale serait l'optimisation naturelle sur des documents d'un ordre de grandeur plus denses.

2.3.4 Modèle de coût : tokenisation visuelle et effet du tuilage

Le tuilage guidé par le DOM produit davantage d'unités que la grille aveugle (section 3.2.3), et rien ne garantit d'avance que cela coûte moins cher à l'encodeur qui les consomme. Le modèle de coût qui suit tranche la question.

Un modèle vision-langage à tour de vision de type *Vision Transformer* [8] découpe une image en *patches* carrés de côté fixe p et les traite comme une séquence de tokens par auto-attention [35]. Pour une image de dimensions $w \times h$, le nombre de tokens visuels produits est, à un facteur de regroupement près,

$$n_{\text{tokens}}(w, h) \approx \frac{w \cdot h}{p^2} = \frac{\text{aire}(\mathcal{B})}{p^2}, \quad (2.5)$$

soit une quantité proportionnelle à l'aire de la boîte recadrée et indifférente à sa forme. Un recadrage deux fois plus étroit à hauteur égale coûte donc deux fois moins de tokens. Chaque pixel de $\Delta A(v)$ épargné par la mesure serrée est ainsi un token visuel que l'encodeur n'a jamais à calculer, sans perte d'information puisque ce pixel était du fond blanc.

Cette proportionnalité n'est pas illimitée, le processeur d'image bornant l'aire encodée entre un plancher $A_{\min}$ et un plafond $A_{\max}$. Le coût réel suit donc une fonction d'écèlement.

$$n_{\text{tokens}}^{\text{écèlement}}(\mathcal{B}) = \frac{\text{clamp}(\text{aire}(\mathcal{B}); A_{\min}, A_{\max})}{p^2}, \quad (2.6)$$

$$\text{clamp}(a; a_{\min}, a_{\max}) = \min(\max(a, a_{\min}), a_{\max})$$

Comparer deux stratégies de découpage sur leur aire totale de pixels ne revient à les comparer sur leur coût en tokens qu'à la condition que leurs unités respectent $A_{\min} \leq \text{aire}(\mathcal{B}) \leq A_{\max}$. C'est une condition à vérifier plutôt qu'une propriété acquise.

Pour une page découpée en tuiles $T = \{t_1, \dots, t_M\}$, le coût total d'encodage vaut $\mathcal{N}(T) = \sum_{i=1}^M n_{\text{tokens}}^{\text{écèlement}}(t_i)$. Deux propriétés de cette somme rendent un découpage plus fin favorable sur cette famille de modèles.

Tant qu'aucune unité n'atteint $A_{\max}$, $\mathcal{N}$ ne dépend que de l'aire totale occupée, et non du nombre d'unités qui la composent. Produire davantage d'unités plus petites ne coûte alors pas plus cher tant que l'aire totale utile ne croît pas.

Tant qu'aucune unité ne descend sous $A_{\min}$, aucune ne paie de coût plancher. Cette seconde propriété tient à la résolution dynamique de Qwen2-VL [36], employé ici. Sous la résolution canonique fixe des générations antérieures, chaque unité payait le même coût quelle que soit sa taille réelle, si bien que le mécanisme exploité ici y serait inopérant.

Pour Qwen2-VL, le côté effectif de *patch* après regroupement vaut 28 px, et le plafond a été porté à $A_{\max} = 12\,845\,056$ px, douze fois le défaut de la bibliothèque (chapitre B). La plus grande unité que la chaîne puisse produire est une tuile carrée de 2048 pixels de côté, soit 4 194 304 px, et la bande de grille en fait 2 097 152 px. Aucune n'atteint le plafond,

donc aucune n'est écrêtée et la condition ci-dessus est vérifiée. Le tableau 2.1 applique les équations (2.5) et (2.6) aux unités des deux conditions.

Tableau 2.1 – Aire et nombre de tokens visuels des unités produites par les deux conditions de l'évaluation A

Unité	Aire (px)	Tokens visuels	
		sans écrêtage	sous 1,00 Mpx
Tuile DOM, moyenne	157 170	200	200
Bande de grille, 2048×1024	2 097 152	2 675	1 280
Tuile DOM maximale, 2048^2	4 194 304	5 350	1 280
Par page, tuilage DOM (24,0 unités)	3,79 M	4 810	4 810
Par page, grille (4,9 unités)	10,17 M	13 100	6 270
Rapport grille / DOM	2,7	2,7	1,3

Le facteur 2,7 de la section 3.2.3 est donc un rapport de tokens autant que de pixels. La dernière colonne montre ce qu'il deviendrait au plafond par défaut, où la bande de grille serait écrêtée quand la tuile DOM ne le serait pas, ce qui ramènerait le gain à 1,3. L'amplitude du gain dépend donc d'un paramètre de configuration, ce dont une reproduction doit tenir compte.

2.3.5 Découpage en sections des pages démesurées

Une capture pleine page peut atteindre plusieurs dizaines de milliers de pixels de hauteur, 46 000 pour l'article consacré aux Beatles, dont la discographie et la liste de récompenses dominent la page. À cette échelle, la génération séquentielle de questions dépasse plusieurs heures par page, sans point de reprise intermédiaire.

Exclure ces pages serait contraire à l'objectif du projet, qui vise les documents longs. Elles sont donc découpées en amont du tuilage. Une page dont la hauteur dépasse dix fois la hauteur maximale de tuile, soit 20 480 px, est scindée aux frontières de titres h1 et h2. Le contenu précédant le premier titre forme une section « préambule » sans titre. Une section restant trop haute à elle seule, un h2 suivi d'un tableau de discographie géant, est retranchée par un budget de hauteur fixe de 20 480 px. C'est le compromis de la section 2.3.3 appliqué un cran au-dessus, entre éléments plutôt qu'à l'intérieur d'un seul.

Chaque section est ensuite traitée par le reste de la chaîne comme une page à part entière, avec son propre nœud page et sa propre unité de travail sauvegardée indépendamment. Le seuil est franchi par 12 des 19 articles du corpus, qui produisent 185 unités de page (tableau 3.1). Le découpage est donc le régime courant sur ce corpus, non un filet de sécurité. L'évaluation A le désactive et rend chaque article comme une unité unique, faute de quoi les conditions comparées ne porteraient pas sur la même étendue de document (section 2.8.2).

Tableau 2.2 – Les sept relations du graphe multigranulaire et leur densité mesurée (la dernière colonne donne la médiane des arêtes par unité de page)

Relation	Domaine	Définition	Arêtes
contains	page → élém.	rattachement plat de chaque tuile à sa page	6
reading_order	élém. → élém.	successeur dans l'ordre de lecture spatial	5
layout_adjacency	élém. ↔ élém.	voisinage 2D (gauche, droite, dessus, dessous)	8
section_hierarchy	titre → contenu	arbre des sections respectant l'imbrication h1–h4	4
semantic_neighbor	élém. ↔ élém.	proximité lexicale TF-IDF au-delà d'un seuil (désactivée par défaut)	0
links_to	élém. → page	lien hypertexte vers un autre document	8
continues	page → page	section suivante d'une page découpée	1

Tiles and reading order, overlaid on the page

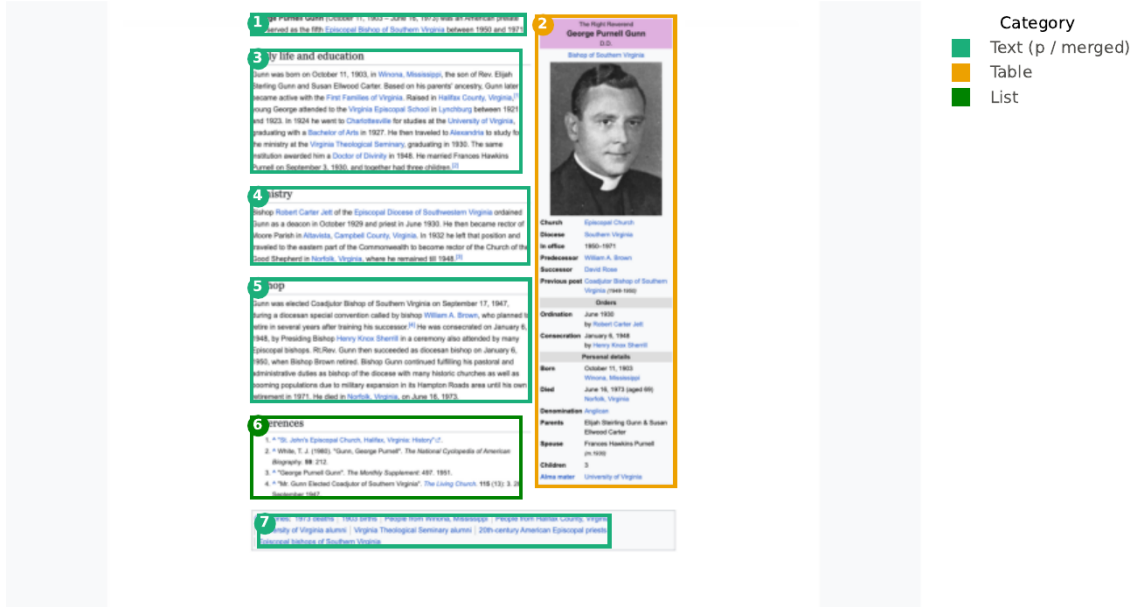


Figure 2.3 – Tuiles produites sur en.wikipedia.org/wiki/George_P._Gunn, colorées par catégorie de contenu et numérotées dans l'ordre de lecture calculé

2.4 Construction du graphe multigranulaire

Les tuiles sont organisées en un graphe orienté multi-arêtes $G = (V, E)$, avec $V = V_{\text{page}} \cup V_{\text{élé}}m$ un nœud `page` et un nœud `élément` par tuile, et $E = \bigcup_{r \in \mathcal{R}} E_r$ l'union de sept relations typées, $E_r \subseteq V \times V$. Cinq d'entre elles, $\mathcal{R}_{\text{MAGE}} \subset \mathcal{R}$, reprennent le graphe de MAGE-RAG [45]. Les deux autres, $\{\text{links_to}, \text{continues}\} = \mathcal{R} \setminus \mathcal{R}_{\text{MAGE}}$, répondent à un besoin propre au contexte web (section 2.4.5). Le tableau 2.2 les récapitule. Le contrôleur ne propage que le long de trois d'entre elles, $\mathcal{R}_{\text{parcourue}} = \{\text{reading_order}, \text{links_to}, \text{continues}\}$, ce que discutent la section 3.4 et la section A.5.

2.4.1 Ordre de lecture spatial

L'ordre de lecture n'est pas l'ordre du DOM, pour la raison établie en section 2.1.4. Il est calculé par une heuristique spatiale sur les coordonnées mesurées, dans la lignée des travaux d'analyse de mise en page qui traitent l'ordre de lecture comme une prédiction plutôt que

comme une lecture de l'arbre [38]. Les tuiles sont d'abord regroupées en rangées. Partant de la tuile la plus haute non encore assignée, toutes les tuiles dont l'ordonnée est à moins de 20 px de la sienne forment une rangée. Chaque rangée est ensuite triée de gauche à droite, et les rangées sont concaténées de haut en bas.

La tolérance de 20 px est le seul paramètre libre de l'heuristique. Elle absorbe les décalages de ligne de base entre un titre et un élément flottant voisin sans fusionner deux rangées distinctes.

2.4.2 Voisinage de mise en page

L'ordre de lecture aplatit la structure 2D en une chaîne, ce qui rend deux tuiles côte à côte indiscernables de deux tuiles empilées. La relation `layout_adjacency` restitue ce voisinage en réutilisant le même regroupement en rangées. Elle produit deux types d'arêtes.

- Les voisins gauche et droite, entre tuiles consécutives d'une même rangée.
- Les voisins dessus et dessous, entre une tuile et la tuile de la rangée suivante qui la recouvre horizontalement. En cas de plusieurs candidates, celle dont le centre horizontal est le plus proche est retenue.

Chaque arête est insérée dans les deux sens, car la règle du centre le plus proche appliquée en sens inverse ne désignerait pas nécessairement la même voisine. La symétrie est une décision de construction et pas une propriété de la règle.

Le second type capture les voisinages que l'ordre linéaire fait diverger. Le bas d'une colonne de gauche et le haut de la colonne de droite sont consécutifs dans l'ordre de lecture mais visuellement éloignés, et inversement.

2.4.3 Arbre des sections

La relation `section_hierarchy` construit un arbre de sections, distinct du rattachement plat de `contains`. Un parcours de l'ordre de lecture maintient une pile de titres ouverts, chacun associé à son niveau. À la rencontre d'un titre de niveau ℓ , la pile est dépilée jusqu'à ce que son sommet ait un niveau strictement inférieur à ℓ ; une arête est créée depuis ce sommet vers le nouveau titre, qui est empilé. Une tuile de contenu est rattachée au sommet courant. Un `h3` est ainsi rattaché au dernier `h2` ouvert, non au dernier `h1`, et le contenu précédant le premier titre ne produit aucune arête.

L'appariement de la section 2.3.3 interagit avec ce parcours. Une tuile de balise composée `h2+p` est traitée comme un titre de niveau 2 — c'est le premier composant qui porte le niveau — et comme le contenu de cette même section, qui n'engendre donc pas d'arête vers elle-même. En pratique l'arbre est moins dense qu'avec des titres isolés : la relation n'apparaît qu'entre titres, et vers les tuiles de contenu qui suivent une section déjà ouverte sans porter elles-mêmes de titre. La densité mesurée figure au tableau 2.2; la section A.5 discute ce que le contrôleur en tire.

2.4.4 Voisinage sémantique

La cinquième relation de MAGE-RAG relie des éléments proches sémantiquement, indépendamment de leur position. Elle est implémentée par similarité cosinus sur vecteurs TF-IDF entre les aperçus textuels des tuiles, avec un seuil de 0,2.

Deux garde-fous l'encadrent. Une sélection gloutonne borne le degré afin que les paires au-delà du seuil soient triées par similarité décroissante, puis retenues tant qu'aucune des deux tuiles n'a atteint trois voisins sémantiques. Cela évite un graphe excessivement dense sur une page au contenu répétitif comme une discographie ou un tableau de résultats sportifs. Cette relation est désactivée par défaut pour servir de condition d'ablation. Cette comparaison

n’a pas été conduite ici (section A.5), et toutes les mesures du chapitre 3 portent donc sur des graphes à six relations sur sept. La relation inactive est celle que ce travail hérite de MAGE-RAG. Les deux relations qui lui sont propres sont actives, et ce sont elles qui produisent l’effet mesuré en section A.2.

2.4.5 Relations inter-documents

Les deux relations ajoutées répondent à un besoin que MAGE-RAG n’a pas, puisqu’il opère sur un document unique déjà délimité.

links_to. Cette relation relie un nœud élément à une page externe, à partir des liens extraits en section 2.2. Un second budget s’y applique, puisque seules les $k_{\text{liens}} = 3$ premières tuiles porteuses de liens, dans l’ordre de lecture de l’unité de page, voient leurs liens conservés. Sur une page non découpée, l’ordre de lecture commence par l’introduction de l’article, et ce budget retient donc les liens qui définissent le sujet. Sur une section issue du découpage de la section 2.3.5, il retient les premiers liens de cette section, sans garantie de saillance comparable. Au-delà, les liens sont ignorés pour borner le facteur de branchement du parcours de corpus.

continues. Cette relation chaîne les sections d’une page découpée dans leur ordre d’origine. Elle est sémantiquement distincte de **links_to**, un lien menant à un autre document quand une continuation mène à la suite du même. Le contrôleur les traite néanmoins symétriquement, puisque dans les deux cas franchir l’arête signifie sortir du périmètre de la page courante.

2.4.6 Graphe de corpus multi-pages

À partir d’une page d’amorce, le système parcourt jusqu’à $m = 3$ pages atteignables par ses arêtes **links_to**. Le parcours est d’un seul saut et non récursif, car suivre les liens de proche en proche ferait croître le nombre de pages exponentiellement, pour un bénéfice incertain sur des questions qui portent rarement à plus d’un saut du sujet (section A.5).

Chaque page est traitée indépendamment par la même chaîne, puis les graphes sont fusionnés. Deux mécanismes rendent la fusion possible.

Le premier est le préfixage par espace de noms. Chaque page utilise indépendamment les identifiants `page`, `tile_0000` et suivants. Tous les nœuds d’un sous-graphe sont donc préfixés avant fusion par un identifiant court dérivé de son URL, ce qui garantit l’unicité sans coordination.

Le second est le point d’entrée. Le graphe d’une page expose un nœud page canonique comme cible d’un lien entrant, le nœud unique si la page n’a pas été découpée et la première section sinon. Lors de la fusion, chaque nœud `external_page` tenant lieu de substitut est remplacé par le point d’entrée réel. Ce recâblage transforme une collection de graphes de pages en un graphe de corpus navigable, et rend possible l’expansion inter-documents du contrôleur (section 2.6.3).

2.4.7 Rendu structuré pour le lecteur

Un sous-graphe en portée pour une requête peut être sérialisé en XML, dans l’esprit de l’étape de rendu structuré multimodal de MAGE-RAG. Un élément `<page>` porte l’URL et le titre, et contient un élément `<element>` par tuile directement rattachée, avec son identifiant, sa balise DOM, son état courant et son texte. Ce format donne au modèle lecteur une vue structurée de ce qui a été retenu et de ce qui a été écarté, plutôt qu’une concaténation de fragments sans provenance. Il est employé par l’application de démonstration (section 2.7),

```

1 <page url="en.wikipedia.org/wiki/George_P._Gunn" title="George P. Gunn">
2   <element id="tile_0000" tag="merged" state="inactive">...</element>
3   <element id="tile_0003" tag="p" state="opened">
4     Gunn was born on October 11, 1903, in Winona, Mississippi, ...
5   </element>
6   <element id="tile_0004" tag="h2+p" state="pruned">...</element>
7   ...
8 </page>

```

Extrait 2.1 – Rendu XML d’un graphe de page après passage du contrôleur pour la requête « When and where was George Purnell Gunn born? »

tandis que les quatre évaluations du chapitre 3 transmettent les preuves sous la forme concaténée de la section 2.6.5.

Les identifiants `tile_NNNN` suivent l’ordre de production du tuilage, qui trie les groupes par position. Il coïncide avec l’ordre de lecture de la section 2.4.1 sauf lorsque deux tuiles d’une même rangée visuelle ont des ordonnées distinctes.

2.5 Entraînement contrastif du lecteur

Un modèle vision-langage générique sait décrire une tuile, mais rien ne le prépare à juger si elle permet de répondre à une question donnée. Il y est entraîné par contraste, en quatre temps. Les données sont d’abord construites, par génération synthétique de paires question-tuile puis minage de négatifs difficiles. Le modèle génératif est ensuite converti en encodeur, puis adapté par LoRA. Une perte InfoNCE est enfin optimisée sur les plongements obtenus.

2.5.1 Génération synthétique de paires question-tuile

Pour chaque tuile retenue, l’image seule est présentée au modèle vision-langage d’inférence (qwen3-v1, chapitre B), avec la consigne de formuler une question factuelle et autonome dont la réponse est visible dans cette image et uniquement dans cette image. L’invite exige une question spécifique, portant sur un nombre, un nom, une date, une définition ou une légende, et une réponse impossible à deviner sans avoir vu la tuile. Si la tuile ne contient aucune information exploitable, le modèle répond exactement SKIP. La réponse attendue est un objet JSON strict, ce qui rend l’analyse syntaxique déterministe. L’invite complète figure en chapitre C.

C’est la transposition à des tuiles d’image de la recette de génération synthétique de requêtes établie par InPars [2] et Promptagator [4].

Toutes les tuiles retenues sont sauvegardées sur disque, y compris celles ayant reçu SKIP, car une telle tuile reste un candidat valide comme négatif difficile pour une question portant sur une autre tuile de la même page. Sur le corpus, 67 tuiles sur 1 784 reçoivent SKIP (tableau 3.1).

2.5.2 Minage de négatifs difficiles

Pour chaque question, toutes les autres tuiles de la même page sont classées par similarité cosinus TF-IDF avec la question, et les $K = 2$ meilleures qui franchissent un filtre anti-faux-négatif sont retenues. Le scoreur lexical est choisi pour son coût, puisqu’il ne demande ni GPU ni modèle à charger alors que le minage s’exécute sur des milliers de tuiles. Sa configuration est reportée en chapitre B.

Le filtre anti-faux-négatif opère en deux temps, de coût très différent.

Le premier est lexical et gratuit. Tout candidat dont l’aperçu textuel contient déjà la réponse verbatim est rejeté, ce qui attrape les faux négatifs littéraux, par exemple deux blocs mentionnant la même date.

Le second, optionnel, est un juge vision-langage. Le filtre lexical ne détecte que les recouvrements textuels et laisse passer un candidat exprimant la même information par un synonyme, ou visuellement, comme une couleur dans un graphique, un logo ou une valeur lue sur un axe. L’image candidate est alors soumise au modèle avec la question et la réponse attendue. Ce filtre coûte un appel par candidat non écarté en amont. Il est désactivable, et les jeux de données du chapitre 3 sont construits avec le filtre lexical seul.

Le minage conserve une question même lorsque moins de K candidats franchissent le filtre, et l’exigence de K négatifs n’est appliquée qu’au chargement du jeu de données pour l’entraînement. Sur les 1 339 exemples construits, 1 282 portent les deux négatifs attendus, 27 n’en portent qu’un et 30 aucun (tableau 3.1). Ces 57 derniers sont écartés au chargement, après le partitionnement de la section 2.5.7.

Le minage opère au sein d’une même page, les identifiants de tuiles n’étant uniques qu’à cette échelle. L’étendre au corpus exigerait de reprendre en amont le préfixage par espace de noms de la section 2.4.6. Cette extension est mise en œuvre pour le chemin HotpotQA (section 2.8.4), où les positifs proviennent de deux pages et où elle est obligatoire.

2.5.3 Du modèle génératif à l’encodeur

Le modèle affiné, Qwen2-VL-7B-Instruct [36], est génératif. Il transforme du texte et des images en texte et n’expose aucun vecteur de représentation, alors qu’une perte contrastive en exige un.

La technique PromptEOL [17] lève cet obstacle. L’entrée est suivie d’une courte instruction fixe, reproduite en chapitre C, et l’état caché de la dernière couche à la position du dernier token est pris comme plongement. L’instruction contraint le modèle à comprimer le sens de l’entrée en une position unique.

Le point qui rend la technique adaptée ici est qu’elle est indifférente à la modalité. Le plongement d’une question textuelle et celui d’une image de tuile sont calculés de la même manière, avec la même instruction, seule l’entrée qui la précède changeant. Question et tuile vivent donc par construction dans le même espace, sans tête de projection additionnelle à apprendre. En notant f_θ cette fonction d’encodage, $\mathbf{z}_q = f_\theta(q)$ pour une question et $\mathbf{z}_u = f_\theta(u)$ pour une tuile u , la perte contrastive de la section 2.5.5 porte exclusivement sur θ , à travers les LoRA appliqués aux deux tours (section 2.5.4).

Où se joue la compréhension visuelle. La tour de vision de f_θ reçoit une séquence de *patches* (équation (2.5)). Pour une tuile issue de la grille aveugle, qui chevauche potentiellement plusieurs sections hétérogènes (section 3.2.2), cette séquence mélange les *patches* de plusieurs entités sans qu’aucune limite ne les sépare, et le gradient de l’équation (1.2) doit pousser f_θ à extraire un signal discriminant d’une représentation dont une partie, potentiellement majoritaire, est hors sujet. Pour une tuile issue du tuilage guidé, chaque *patch* appartient par construction à l’unique élément sur lequel porte la question, et le signal à amplifier occupe toute la séquence. f_θ reste un Qwen2-VL générique, sans mécanisme d’attention ni architecture d’encodeur nouvelle. C’est la frontière de recadrage, déplacée en amont de l’encodeur, qui change la nature de la difficulté, de juger un mélange de contenus à juger un contenu homogène. La section 3.3 mesure l’effet de l’entraînement contrastif sur des tuiles ainsi alignées, mais ce travail ne mesure pas séparément ce que l’alignement DOM apporte à l’entraînement lui-même, faute d’un entraînement identique mené sur des tuiles de grille.

Chemin d'exécution et modèle affiné. L'affinage n'emprunte pas le chemin utilisé pour la génération de questions et la lecture des tuiles (sections 2.5.1 et 2.6.5), qui s'adresse à un serveur Ollama n'exposant qu'une API de génération, sans accès aux poids ni aux gradients. Il charge le modèle via `transformers` et `peft`, ce qui exige un accès complet aux poids et un GPU loué. Le modèle affiné n'est donc pas celui de l'inférence, et Qwen2-VL-7B-Instruct a été retenu pour la stabilité de sa classe `peft` au moment de ce travail, quand Qwen2.5-VL et Qwen3-VL l'étaient moins pour l'affinage par LoRA. C'est un choix d'outillage, non une revendication d'équivalence entre les deux modèles. Cette proximité de famille sans identité est reprise en section A.5 comme risque de biais à contrôler.

2.5.4 Adaptation LoRA sur la tour de vision et le décodeur

L'adaptation utilise LoRA [14] avec un rang $r = 16$, un facteur d'échelle $\alpha = 32$ et un *dropout* de 0,05, soit 44 302 336 paramètres entraînaibles sur les sept milliards du modèle de base.

Cibler les deux tours. LoRA est appliqué aux projections d'attention du décodeur (`q_proj`, `k_proj`, `v_proj`, `o_proj`) et à celles de la tour de vision (`qkv`, `proj`). Juger si une tuile répond à une question exige d'affiner simultanément ce qui est vu dans l'image et ce qui est compris de la question. Geler la tour de vision reviendrait à demander au décodeur de compenser seul une représentation visuelle inadaptée à la tâche de récupération. La section 1.5.2 situe ce choix dans les variantes multimodales de LoRA.

Découvrir les modules cibles par introspection. Les noms de modules internes de Qwen2-VL varient d'une version de `transformers` à l'autre. Seuls des suffixes sont donc spécifiés, et le code parcourt le modèle chargé pour découvrir les modules linéaires correspondants. LoRA n'est appliqué qu'aux modules effectivement trouvés, ce qui rend l'adaptation robuste aux changements d'architecture interne mais impose de journaliser la liste obtenue à chaque exécution.

2.5.5 La perte contrastive

Soit un lot de $B = 4$ questions, chacune accompagnée d'une tuile positive et de $K = 2$ négatifs difficiles minés. Les plongements des B questions et des $B(1 + K)$ tuiles sont calculés, et la perte minimisée est l'InfoNCE de l'équation (1.2) étendue au lot entier,

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(\text{sim}(\mathbf{z}_{q_i}, \mathbf{z}_{u_i^+})/\tau)}{\sum_{j=1}^B \left[\exp(\text{sim}(\mathbf{z}_{q_i}, \mathbf{z}_{u_j^+})/\tau) + \sum_{l=1}^K \exp(\text{sim}(\mathbf{z}_{q_i}, \mathbf{z}_{u_{j,l}^-})/\tau) \right]}, \quad (2.7)$$

avec une température $\tau = 0,05$. Le dénominateur porte sur les $B(1 + K)$ tuiles du lot et non sur les $1 + K$ tuiles associées à la question courante, si bien que chaque positif entre en compétition à la fois avec ses propres négatifs minés et avec les tuiles de toutes les autres questions du lot, les *in-batch negatives* de la section 1.5.1. À $B = 4$ et $K = 2$, le nombre de négatifs effectifs par question,

$$N_{\text{eff}} = B(1 + K) - 1 = 11, \quad (2.8)$$

passé de 2, les seuls négatifs minés si l'on entraînait avec $B = 1$, à 11, sans plongement supplémentaire à calculer. Le coût du lot croît linéairement en B , le nombre de comparaisons quadratiquement.

Lecture de l'équation (2.7) comme classification. Pour une question q_i fixée, notons $\mathbf{s}_i \in \mathbb{R}^{B(1+K)}$ le vecteur des similarités $\text{sim}(\mathbf{z}_{q_i}, \cdot)/\tau$ contre les $B(1+K)$ tuiles du lot, et y_i l'indice de la tuile positive u_i^+ dans ce vecteur. Le terme sommé de l'équation (2.7) est alors exactement l'entropie croisée catégorielle $-\log \text{softmax}(\mathbf{s}_i)_{y_i}$. Apprendre par contraste revient donc à résoudre, pour chaque question du lot, un problème de classification à $B(1+K)$ classes dont une seule est correcte. Le gradient par rapport à $\mathbf{z}_{q_i}$ se décompose en une force qui rapproche $\mathbf{z}_{q_i}$ de $\mathbf{z}_{u_i^+}$ et des forces qui l'éloignent de chaque négatif u , chacune pondérée par la probabilité $\text{softmax}(\mathbf{s}_i)_u$ que le modèle attribue déjà à ce négatif. Un négatif que le modèle confond encore avec le positif reçoit ainsi un gradient plus grand qu'un négatif déjà écarté avec confiance.

La température τ contrôle l'acuité de cette pondération. En la faisant tendre vers 0, le softmax se concentre sur l'argument maximal et le gradient se resserre presque entièrement sur le négatif le plus proche du positif, ce qui accélère l'apprentissage sur les cas difficiles mais amplifie d'autant l'effet d'un faux négatif introduit par le minage (section 2.5.2). À $\tau = 0,05$, ce compromis penche délibérément du côté de la discrimination fine, cohérent avec l'objectif de la section 2.5.1 de produire des questions dont la réponse n'est visible que dans une tuile précise.

2.5.6 Contraintes mémoire

Sur une carte de 80 GB, le chemin de calcul par lots des plongements a d'abord échoué sur un dépassement de mémoire GPU, avec 79,16 GB occupés sur les 79,25 GB disponibles. L'usage est réel et non fragmenté, puisqu'un lot pousse $B(1+K) = 12$ tuiles à travers le modèle simultanément, soit environ 64 000 tokens visuels au total. Aucune de ces tuiles n'étant écrêtée (section 2.3.4), cette longueur croît proportionnellement à l'aire cumulée du lot.

L'activation du *gradient checkpointing*, qui recalcule les activations lors de la passe arrière plutôt que de les conserver, résout le problème au prix de 20 % à 30 % de temps de calcul supplémentaire. C'est le réglage par défaut retenu à cette résolution de tuile.

2.5.7 Partitionnement pour l'évaluation

Un partitionnement au niveau des exemples serait trompeur, car deux tuiles voisines d'une même page produisent des questions fortement corrélées, et un modèle ayant vu l'une en entraînement bénéficierait indûment sur l'autre en validation.

Le partitionnement est donc au niveau de l'article, en proportion 80/20 visée puis arrondie à l'article entier, avec des articles strictement disjoints entre les deux parties. Aucune tuile et aucune question quasi dupliquée ne peut traverser la frontière. L'adaptateur évalué en section 3.3 est entraîné sur la seule partie d'entraînement.

2.6 Contrôleur de preuves en ligne

À la requête, le contrôleur décide quels nœuds élément le lecteur ouvre réellement, plutôt que de lui présenter la page entière ou une sélection top- k figée. Il généralise l'algorithme de contrôle de MAGE-RAG.

2.6.1 Machine à états

Chaque nœud élément porte un attribut d'état, initialisé à INACTIVE lors de la construction du graphe, que le contrôleur fait évoluer à travers la machine à quatre états de la figure 2.4.

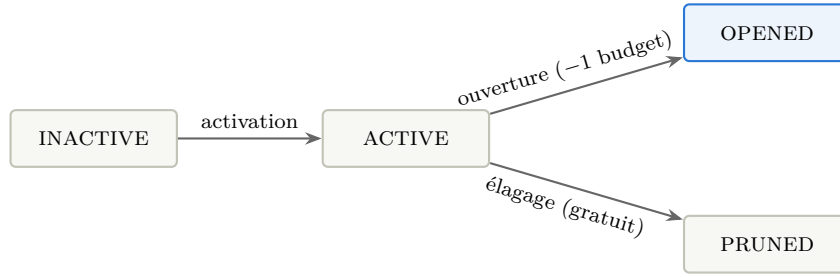


Figure 2.4 – Machine à états du contrôleur. L’ouverture d’un nœud consomme une unité de budget et peut activer ses voisins encore inactifs, le long de `reading_order` sur la même page ou en franchissant une frontière de document via `links_to` et `continues`. L’élagage est gratuit et s’applique à un nœud actif dont le score est sous le seuil d’ouverture, puis à tout nœud resté actif lorsque le budget est épuisé. Un nœud jamais activé reste INACTIVE.

Un nœud INACTIVE n’appartient pas à la frontière courante. Un nœud ACTIVE est candidat à l’ouverture. Un nœud OPENED a été lu et son contenu fait partie des preuves. Un nœud PRUNED a été jugé non pertinent, ou laissé de côté faute de budget.

2.6.2 Politique *best-first* budgétée

Le contrôleur cherche un sous-ensemble $O \subseteq V_{\text{élém}}$ de nœuds à ouvrir sous la contrainte de budget $|O| \leq B$, où B désigne ici le nombre d’ouvertures autorisées et non la taille de lot de la section 2.5.5. Il maximise informellement la pertinence cumulée $\sum_{n \in O} \text{score}(n)$ tout en respectant une contrainte d’accessibilité, un nœud ne pouvant entrer dans O que s’il a d’abord été rendu ACTIVE par l’ouverture d’un voisin déjà dans O , ou par l’amorçage initial.

C’est un problème de sélection sous contrainte de cardinalité et de connexité, pour lequel l’algorithme 2 implémente une heuristique *best-first* gloutonne plutôt qu’une optimisation exacte. Rien ne garantit que O maximise $\sum \text{score}(n)$ sur les sous-graphes connexes de cardinalité B atteignables depuis l’amorce, seulement que chaque décision locale est rationnelle au moment où elle est prise. Ce choix reprend celui de MAGE-RAG et privilégie le calcul en une passe.

La politique comporte trois phases. L’amorçage active les k_{seed} nœuds inactifs les mieux notés pour la requête, seule phase où la sélection est globale. La boucle principale prend répétitivement le nœud actif le mieux noté, l’élague sans consommer de budget si son score est sous le seuil d’ouverture, et sinon l’ouvre, ce qui consomme une unité de budget et rend actifs ses voisins encore inactifs qui franchissent le seuil d’activation. La finalisation élague tout nœud resté actif une fois le budget épuisé, de sorte qu’aucun nœud ne subsiste dans un état transitoire.

Ce qui distingue cette politique d’une sélection *top-k* n’est ni le score ni le budget, identiques, mais le fait que les voisins ne sont considérés qu’après l’ouverture de leur nœud. La frontière est conditionnée par ce qui s’est déjà révélé pertinent plutôt que par un classement global figé.

Effet de l’égalité des deux seuils. Les deux seuils valent 0,05 (chapitre B), et leur égalité restreint l’élagage gratuit. Un nœud activé par un voisin a franchi θ_{act} , donc son score atteint aussi θ_{ouv} et il ne peut pas être élagué gratuitement. Seules les amorces, activées sans test de seuil, le peuvent. Le contrôleur n’ouvre donc moins de B nœuds que pour deux raisons, l’élagage d’une amorce ou l’épuisement de la frontière, et la section A.2.3 mesure 4,86 ouvertures en moyenne pour un budget de 5.

Algorithme 2 — Contrôle de preuves *best-first* sous budget

Entrée: graphe G , requête q , budget B , amorce k_{seed} , seuils θ_{ouv} et θ_{act}

Sortie: liste des nœuds OPENED, en ordre de lecture

```
1 pour chaque nœud élément  $n$  de  $G$  faire
2    $\text{score}(n) \leftarrow \text{sim}(q, \text{texte}(n))$  // TF-IDF
3 activer les  $k_{\text{seed}}$  nœuds inactifs de plus haut score
4  $b \leftarrow 0$ 
5 tant que  $b < B$  et il existe un nœud ACTIVE faire
6    $n \leftarrow \arg \max \text{score}$  parmi les nœuds ACTIVE
7   si  $\text{score}(n) < \theta_{\text{ouv}}$  alors
8      $n \leftarrow \text{PRUNED}$  // gratuit
9     continuer
10   $n \leftarrow \text{OPENED}$  ;  $b \leftarrow b + 1$ 
11   $N \leftarrow \text{VOISINSORDREDELECTURE}(n) \cup \text{VOISINSINTERDOCUMENTS}(n)$ 
12  pour chaque  $n' \in N$  tel que  $n'$  est INACTIVE faire
13    si  $\text{score}(n') \geq \theta_{\text{act}}$  alors
14       $n' \leftarrow \text{ACTIVE}$ 
15 élaguer tout nœud encore ACTIVE
16 return les nœuds OPENED, triés par ordre de lecture
```

2.6.3 Expansion inter-documents

Le contrôleur de MAGE-RAG, et la première implémentation à sa suite, ne propagent l'activation que le long de `reading_order`, relation qui ne franchit jamais une frontière de page. Un nœud ouvert près de la fin d'une page ne peut donc activer aucun nœud d'une page liée ni de la section suivante, alors même que les deux arêtes existent. Sur une question multi-sauts dont les preuves sont réparties sur deux articles, le second n'est atteignable que si l'amorce globale y a déjà placé un nœud, c'est-à-dire exactement ce que fait une sélection top- k statique, et moins bien dès que l'amorce en manque un.

Ce point est la cause diagnostiquée de l'incapacité du contrôleur à dépasser la référence statique sur les questions inter-pages (section A.2.2). À l'ouverture d'un nœud, en plus de ses voisins d'ordre de lecture, sont désormais rendus candidats

- tous les éléments de toute page atteignable depuis ce nœud par une arête `links_to`, la page cible étant, grâce au recâblage de la section 2.4.6, une page réellement construite et non un substitut,
- et tous les éléments de toute page atteignable depuis sa page conteneuse par une arête `continues`.

Dans les deux cas, franchir la porte n'entraîne aucune activation automatique, chaque élément candidat devant franchir le même seuil θ_{act} qu'un voisin d'ordre de lecture. Le mécanisme est strictement symétrique à celui qui existait déjà, seul le voisinage considéré s'élargit, et il reste limité à trois des sept relations du graphe (section 2.4). La section A.2 mesure l'effet de ce correctif.

2.6.4 Score de pertinence et point d'extension

Par défaut, la pertinence est mesurée par similarité cosinus TF-IDF entre la requête et le texte de chaque nœud, avec le même scoreur et la même configuration que le minage de négatifs (section 2.5.2). C'est un substitut destiné à être remplacé.

Le contrôleur expose à cet effet un paramètre de scores précalculés. Si un dictionnaire de scores est fourni, il remplace intégralement le scoreur lexical, un nœud absent recevant un score nul. Ce point d’extension est conçu pour y brancher la similarité du lecteur affiné de la section 2.5, sans toucher à la politique. Le calcul de ces scores, un passage avant du modèle par tuile candidate, reste hors de ce module. La section A.5 explique pourquoi ce branchement n’a pas été effectué et pourquoi il constitue la modification la plus prometteuse restant à faire.

2.6.5 Lecture des tuiles retenues

L’algorithme 2 retourne une liste de nœuds, non des preuves. Les tuiles ouvertes sont ensuite relues par le modèle vision-langage, qui en extrait les informations utiles à la requête. Les preuves finales sont la concaténation, en ordre de lecture, de ces extractions, et remplacent donc l’aperçu textuel qui a servi au scorage.

La politique a une propriété structurelle que la section 2.7.4 exploite. Quel nœud est ouvert ne dépend jamais du contenu lu par le modèle, mais uniquement des scores figés en amont. La simulation complète de la machine à états peut donc être déroulée sans aucun appel au modèle, les lectures n’étant effectuées qu’ensuite, sur une liste déjà connue.

2.7 Implémentation et ingénierie

2.7.1 Architecture modulaire

Le système est écrit en Python, avec un module par étage. Le rendu et l’extraction reposent sur Playwright, le tuilage sur des recadrages Pillow, le graphe sur `networkx`. Le client vision-langage s’adresse à un serveur Ollama, ce qui permet de démarrer avec un seul téléchargement de modèle et sans dépendance à `torch`. Le module d’affinage, seul à dépendre de `torch`, `transformers` et `peft`, n’est pas importé par le reste du système et ne charge ces bibliothèques qu’à l’exécution. La cartographie des modules figure en chapitre D.

Une application web expose la chaîne complète derrière un formulaire « URL plus question ». La requête déclenche l’exécution en direct, puis affiche la réponse synthétisée, le graphe interactif (catégorie de contenu par couleur, état du contrôleur par bordure, arêtes filtrables par relation) et les pages liées, avec un marqueur sur celles portées par une tuile retenue comme preuve. C’est le seul consommateur du rendu XML de la section 2.4.7. Chaque requête recalcule tout, sans corpus préconstruit, et le temps de réponse va de quelques dizaines de secondes à plusieurs minutes selon la page et le budget.

2.7.2 Reproductibilité des appels au modèle

Décodage glouton. Aucune température n’étant fixée, le serveur échantillonnait avec ses réglages par défaut, donc à température strictement positive. Deux appels sur la même image et la même invite pouvaient produire des réponses différentes, y compris une réponse correcte et une abstention sans aucune différence dans les tuiles ouvertes. La température est fixée à zéro, et à entrée fixée la sortie est déterministe.

Fenêtre de contexte. Le modèle d’inférence raisonne avant de répondre, sur 1 500 à 2 500 tokens pour une extraction factuelle sur une tuile. Ce raisonnement s’ajoute à l’invite et aux tokens visuels de la tuile, soit 200 en moyenne et jusqu’à 5 350 pour une tuile de taille maximale (tableau 2.1). Avec la fenêtre de contexte par défaut du serveur, fixée à 4 096 tokens, la somme épuisait la fenêtre avant l’émission de la réponse finale, et le résultat était un contenu vide, silencieusement interprété comme `SKIP` par l’analyseur de réponses. La

fenêtre est portée à 8 192 tokens, ce qui laisse peu de marge dans le cas le plus défavorable. Cette classe d'erreurs, une défaillance d'infrastructure qui se déguise en résultat de modèle légitime, est celle contre laquelle la journalisation des longueurs de séquence protège le mieux.

Cache sur disque. Chaque appel au modèle est mis en cache sur disque, indexé par un condensat du triplet (image, invite, configuration de décodage). Le décodage étant glouton, un appel déjà effectué se rejoue sans coût. Ce cache rend praticable l'itération sur le code d'évaluation, puisque relancer un script après correction d'un défaut ne repaie pas les heures d'appels déjà effectuées.

2.7.3 Sauvegarde par unité de travail

L'état est sauvegardé par unité de page, une unité étant une page ou une section d'une page découpée. Ce grain répond directement au cas de la section 2.3.5, un échec en cours de traitement d'une page démesurée ne coûtant plus que la section en vol.

2.7.4 Concurrency

Deux étapes exploitent la concurrence, avec au plus 4 appels au modèle simultanés, et dans les deux cas la légitimité du parallélisme découle d'une propriété du système.

La génération de questions émet ses appels en parallèle au sein d'une page. Ces appels sont indépendants, chacun ne recevant qu'une image et une invite fixe.

La lecture des tuiles ouvertes fait de même, et c'est ici que la propriété de la section 2.6.5 devient opérationnelle. Puisque la trajectoire d'états ne dépend jamais du contenu lu, la liste des tuiles à lire est entièrement connue avant la première lecture, et ces lectures sont donc mutuellement indépendantes. Un unique client est réutilisé entre fils d'exécution, chaque appel étant une requête HTTP indépendante sans état mutable partagé à protéger.

2.7.5 Tests

La stratégie de test suit une séparation par dépendances, détaillée en chapitre D. Les tests sans dépendance lourde couvrent la logique géométrique, la construction du graphe, le minage de négatifs, l'assemblage du jeu de données, le cache et la concurrence du client, et la compression d'images. Ils s'exécutent sur des cas synthétiques, sans réseau, sans modèle et sans GPU.

Les tests adossés à Playwright vérifient l'extraction DOM de bout en bout contre une page HTML locale fournie avec le dépôt. Deux cas y sont verrouillés. Un élément flottant partageant sa rangée avec du texte fixe le comportement de la mesure serrée de la section 2.2.4. Deux éléments en chevauchement partiel, tous deux conservés, détectent une régression de la section 2.3.1 vers un élagage trop agressif.

2.8 Protocole expérimental

2.8.1 Construction du jeu de données synthétique

Un script pilote la chaîne sur une liste de pages, avec reprise sur échec du téléchargement. Son unité de travail est l'unité de page, et chaque unité est sauvegardée indépendamment dans un fichier au format JSON lignes. Trois fichiers en résultent, un manifeste associant chaque identifiant de tuile à son image, les paires question-réponse générées, et les exemples contrastifs assemblés avec leurs négatifs. Cette reprise ne couvre pas un échec survenant après la génération des questions, dont la section A.5 rapporte l'effet.

Le plafond de 25 tuiles par unité de page (section 2.3) borne le coût de génération. Il borne aussi la portée de l'évaluation A, ce qu'analyse l'section A.5.

2.8.2 Reproduction de la référence en grille

Pour l'ablation du tuilage (section 3.2), un module réimplémente la référence en grille aveugle indépendamment du module de tuilage. La page rendue est tranchée en bandes de 1 024 px de haut à la largeur de rendu, ce qui reproduit la hauteur de tuile documentée par PixelRAG mais non sa largeur, plus étroite (section 1.3.1). Cet écart de largeur est à porter au compte de la comparaison de budget de pixels, pour la raison exposée en section 2.2. Une troisième condition découpe le texte extrait de la même région de page en fenêtres plates de 400 caractères, sans aucune notion de frontière d'élément. C'est la référence textuelle standard que l'introduction critique et que ni PixelRAG ni MAGE-RAG ne mesurent directement.

Les trois conditions sont construites sur des articles rendus avec un seuil de découpage en sections relâché par rapport à celui de la section 2.3.5. Cette évaluation retient une hauteur de page maximale de 25 fois la hauteur maximale de tuile plutôt que 10, soit 51 200 px contre 20 480 px pour le reste du corpus. Ce seuil propre à l'évaluation, indépendant de celui qui protège des coûts de génération le corpus principal, suffit à ce que chaque article soit traité comme une unité de page unique, de sorte que les trois conditions portent sur la même étendue de document. La section 3.2 précise l'exception rencontrée en pratique sur l'article le plus long du corpus.

L'indépendance de ce module vis-à-vis du module de tuilage est un choix méthodologique, puisqu'elle garantit qu'une modification du tuilage guidé ne peut pas altérer silencieusement la référence à laquelle on le compare.

2.8.3 Ablation de résolution

PixelRAG documente la résolution d'image comme levier d'efficacité, sans préciser ni le filtre de ré-échantillonnage ni les résolutions testées. Un couple pleine résolution et résolution réduite est donc construit sur les tuiles de ce travail, par sous-échantillonnage Lanczos à un facteur d'échelle linéaire de 0,5, soit quatre fois moins de pixels. Les images compressées sont écrites sous les mêmes noms dans un dossier miroir, de sorte que le manifeste et les exemples contrastifs restent réutilisables tels quels, seul le dossier racine des images changeant entre les deux expériences.

2.8.4 Questions multi-sauts issues de HotpotQA

La génération synthétique de la section 2.5.1 a un biais structurel, puisqu'elle demande une question dont la réponse est visible dans une seule tuile. Un contrôleur omniscient pourrait donc répondre à n'importe laquelle de ces questions depuis un unique nœud, ce qui ne sollicite jamais le graphe.

Cet écart est comblé avec HotpotQA [41], dont chaque question nomme, via ses phrases de support, deux articles Wikipédia et les phrases exactes nécessaires dans chacun. Ce signal annoté par des humains est réutilisé plutôt que régénéré, selon la procédure suivante.

1. Les titres d'articles cités sont convertis en URL Wikipédia et rendus par la chaîne existante, inchangée.
2. Pour chaque article, les tuiles dont le texte recouvre une phrase de support sont retenues comme positives. Une même question peut donc avoir des tuiles positives sur deux pages distinctes.
3. Les négatifs difficiles sont minés sur le vivier réunissant les tuiles des deux articles, l'extension inter-pages que la section 2.5.2 laissait de côté étant ici obligatoire.

Une question est abandonnée dans deux cas, plutôt que de produire un exemple incomplet. Le premier est un article qui ne se rend plus, renommé ou supprimé depuis le *dump* Wikipédia de 2017 sur lequel HotpotQA est construit. Le second est une phrase de support introuvable dans le rendu actuel, l'article ayant été réécrit.

L'attribution d'une phrase de support à une tuile se fait par recouvrement de tokens filtrés des mots vides, et non par correspondance de sous-chaîne exacte. Celle-ci survit rarement à une décennie de modifications éditoriales, et l'exiger ferait chuter le taux de conservation pour une raison sans rapport avec ce qui est mesuré.

Chapitre 3

Résultats et limites

Ce chapitre rapporte les deux résultats solidement étayés du système décrit au chapitre 2 : le gain de trouvabilité et de coût du tuilage guidé par le DOM (section 3.2), et le gain de l’entraînement contrastif du lecteur (section 3.3). Les composants en aval — contrôleur budgété, système assemblé bout-en-bout —, les coûts de calcul observés et la discussion complète des limites et des directions de travail sont rapportés en détail dans le [Résultats complémentaires](#) (chapitre A) ; ce chapitre n’en retient que la conclusion.

MAGE-RAG est évalué sur LongDocURL [5] et MMLongBench-Doc [26], deux jeux de données constitués de PDF déjà analysés. Ils ne peuvent solliciter un tuilage guidé par le DOM — un PDF n’expose ni boîtes englobantes, ni sélecteurs CSS, ni frontières de titres HTML — et ne sont donc pas repris ici comme comparaison numérique. Toutes nos évaluations portent sur des pages web réelles.

3.1 Jeu de données

Tableau 3.1 récapitule les volumes construits par le protocole de la section 2.8.

Le taux de SKIP (3,8 %) est un premier signal indirect en faveur du tuilage : une tuile guidée par une frontière d’élément contient presque toujours du contenu exploitable. Les 378 questions qui ne produisent pas d’exemple contrastif proviennent de seize unités de page absentes du jeu construit, et non d’un filtrage par question (section A.5). Le taux de conservation des questions HotpotQA (61 %, 91/150) reflète principalement la dérive éditoriale de Wikipédia depuis le *dump* de 2017 sur lequel HotpotQA est construit, plutôt qu’une défaillance du pipeline ; les deux causes d’abandon sont détaillées en section 2.8.4.

3.2 Le tuilage guidé par le DOM double la trouvabilité, à moindre coût

3.2.1 Protocole

Cette évaluation compare trois représentations du *même* contenu de page : nos **tuiles DOM** (section 2.3), une **grille aveugle** de bandes de 1024 px reproduisant la hauteur documentée par PixelRAG [37] (section 2.8.2), et un **texte plat** découpé en fenêtres de 400 caractères sans notion de frontière d’élément. Dix-huit des dix-neuf articles du corpus sont rendus comme unité de page unique ; les trois représentations sont construites sur cette unité, et 409 questions — celles dont la page n’a pas été découpée en sections — sont classées contre chacune par similarité cosinus TF-IDF, le *même* scoreur dans les trois conditions, pour isoler l’effet du découpage. Le dix-neuvième article (Beatles, le plus long du corpus) est exclu pour une raison opérationnelle : rendu comme page unique il dépasse la mémoire disponible au traitement. Une question n’est évaluée dans une condition que si sa réponse apparaît verbatim dans l’une de ses unités ; c’est ce qui produit le *taux d’attachement* ci-dessous. Le protocole complet — notamment le plafond de 25 tuiles par page et la restriction des trois conditions à une région de page identique, nécessaires à la validité de la comparaison — est détaillé en section A.1.1.

Tableau 3.1 – Volumes du jeu de données utilisé par les évaluations. Les tuiles sans question exploitable sont celles pour lesquelles le modèle a répondu **SKIP** ; elles restent disponibles comme négatifs difficiles (section 2.5.1).

Grandeur	Valeur
Articles Wikipédia distincts	19
dont découpés en sections (section 2.3.5)	12
Unités de page (articles + sections)	185
Tuiles extraites	1 784
Paires question-réponse générées	1 717
tuiles sans question exploitable (SKIP)	67 (3,8 %)
Exemples contrastifs (question + négatifs difficiles)	1 339
questions perdues entre génération et minage	378 (16 unités)
exemples portant les $K = 2$ négatifs attendus	1 282
Questions inter-pages construites depuis HotpotQA	91 / 150 échantillonnées

Les trois viviers de candidats diffèrent fortement en taille (24,0 tuiles DOM par page en moyenne, contre 4,9 bandes de grille et 12,3 fenêtres de texte) : le Recall@ k brut y est donc gonflé mécaniquement par un vivier plus petit (section 1.1.1), et la comparaison de classement qui suit en section A.1.2 utilise le rang percentile invariant à la taille du vivier plutôt que le Recall brut.

3.2.2 Résultat 1 : le taux d’attachement

Le tuilage guidé par le DOM porte la réponse dans une unité récupérable pour 51,8 % des questions (212 sur 409), contre 26,4 % pour la grille aveugle (108 sur 409) — soit presque le double.

L’explication est mécanique. Une bande de 1 024 px découpée sans égard au contenu agrège plusieurs sections hétérogènes ; une réponse courte — une date, un nombre, un nom propre — s’y trouve noyée dans un volume de texte sans rapport, au point que le critère d’attribution ne la retrouve plus. Une tuile alignée sur une frontière d’élément contient au contraire une unité de sens complète et délimitée. Ce résultat détermine un *plafond* : une question dont la réponse ne survit dans aucune unité est perdue quelle que soit la qualité du récupérateur en aval, et doubler le taux d’attachement double donc le nombre de questions auxquelles le système a la *possibilité* de répondre. Le texte plat obtient un taux encore légèrement supérieur, 56,7 % — attendu, puisque le critère d’attribution est lui-même une correspondance de chaîne exacte, structurellement plus favorable à une condition qui manipule le texte directement (section A.1.2).

3.2.3 Résultat 2 : le budget de pixels

Notre tuilage produit environ cinq fois *plus* d’unités que la grille (24,0 contre 4,9 par page). On attendrait un coût de représentation supérieur ; c’est l’inverse qui se produit.

*Le total des pixels par page est de 3,79 millions pour le tuilage DOM contre 10,17 millions pour la grille, soit **2,7 fois moins**, alors même que le tuilage DOM produit cinq fois plus d’unités.*

L’explication est la mesure serrée des balises de texte coulant (section 2.2.4, équation (2.2)) : une bande de grille occupe par construction toute la largeur de rendu quel que soit son contenu, tandis qu’une tuile de texte DOM est recadrée sur la largeur que ses

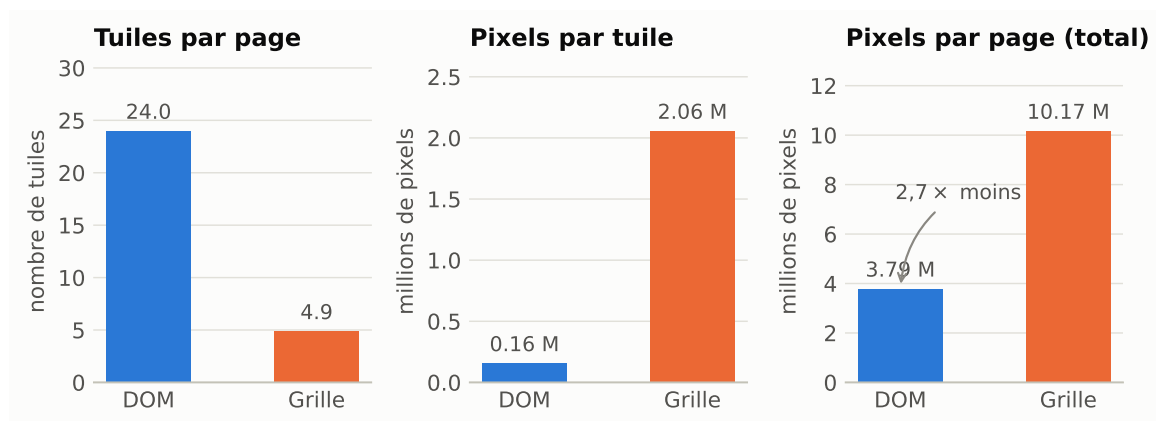


Figure 3.1 – Coût de représentation par page, moyenné sur les 18 pages de l’évaluation, pour la même région de page dans les deux conditions. Le tuilage DOM produit près de cinq fois plus de tuiles, mais chacune est treize fois plus petite, de sorte que le total par page est 2,7 fois inférieur.

lignes occupent réellement — 157 170 pixels en moyenne contre 2 058 000 pour la grille, un rapport de treize qui compense largement le rapport de cinq sur le nombre d’unités. Le nombre de tokens visuels croît avec l’aire de l’image sous réserve du plafond du processeur (section 2.3.4) : sans écrêtage le rapport de 2,7 se transpose à l’identique au coût en tokens, sous le plafond par défaut il tombe à 1,3 (tableau 2.1) — dans les deux cas un coût *inférieur* pour davantage de contenu trouvable, et les deux axes vont dans le même sens.

Deux gains sont donc établis sans ambiguïté : le taux d’attachement (51,8 % contre 26,4 %) et le budget de pixels (2,7 fois moins). Un troisième résultat, plus nuancé — l’avantage de rang percentile du tuilage DOM sur la grille est réel mais modeste, et s’inverse face au texte plat sur le sous-ensemble strictement apparié — est analysé en détail en section A.1.2 : nous y montrons que ce résultat est cohérent avec le scoreur lexical employé, qui n’exploite aucune information spatiale, et non avec un défaut du tuilage.

3.3 L’entraînement contrastif du lecteur fait presque doubler le Recall@1

3.3.1 Protocole

Cette évaluation mesure l’apport de l’entraînement contrastif (section 2.5) sur un partitionnement au niveau de l’**article**, en proportion 80/20, avec des articles strictement disjoints entre entraînement et validation (section 2.5.7). Chaque question de validation est classée contre *toutes* les tuiles de sa page — un scénario de récupération réaliste, non le petit vivier de négatifs minés vu à l’entraînement — avec et sans l’adaptateur chargé sur le *même* modèle de base, en le désactivant plutôt qu’en chargeant un second modèle. Sur 338 questions de validation, 330 sont retenues après exclusion des pages à moins de deux tuiles ou à tuile positive absente du manifeste.

3.3.2 Résultat et analyse

Le gain est net et sans ambiguïté : le Recall@1 passe de 0,433 à 0,785, un quasi-doublement, et le MRR de 0,578 à 0,856. Le partitionnement au niveau de l’article et l’évaluation sur toutes les tuiles de la page écartent les deux mécanismes de fuite les plus courants : le gain reflète donc un transfert réel à des articles jamais vus.

Tableau 3.2 – Modèle de base et modèle affiné par LoRA contrastif, sur la partie de validation (330 questions, articles jamais vus à l’entraînement). Le classement porte sur toutes les tuiles de la page de la question.

Scoreur	R@1	R@3	R@5	MRR
TF-IDF (lexical, référence)	0,461	0,618	0,724	0,579
Modèle de base (sans LoRA)	0,433	0,645	0,752	0,578
Modèle affiné (LoRA contrastif)	0,785	0,915	0,952	0,856

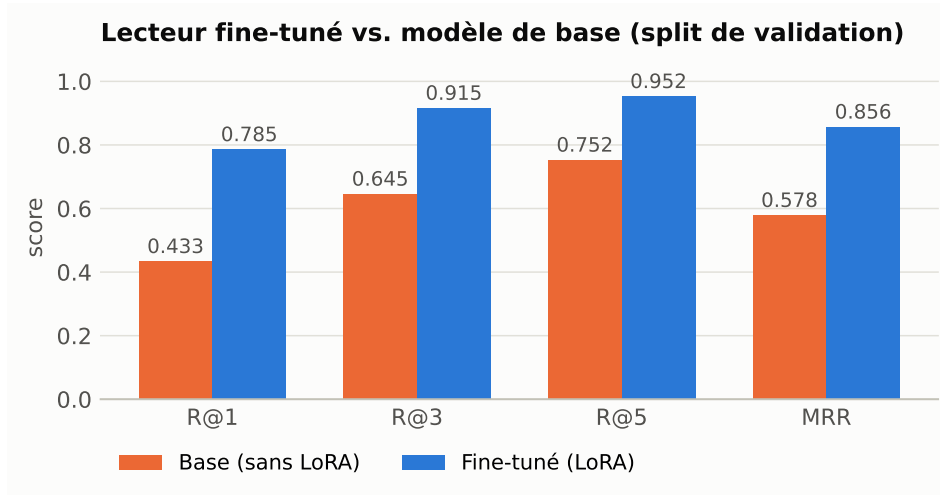


Figure 3.2 – Gain de l’entraînement contrastif sur la partie de validation. Le Recall@1 passe de 0,433 à 0,785 et le MRR de 0,578 à 0,856, sur un partitionnement par article strictement disjoint.

La comparaison avec TF-IDF n’est pas une victoire uniforme du scoreur lexical : sur le Recall@1 et le MRR — les métriques les plus pertinentes pour une sélection à un seul candidat — le modèle vision-langage *non adapté* est à parité ou légèrement en retrait (MRR 0,578 contre 0,579), et ne devance TF-IDF que sur Recall@3/Recall@5. Le gain de Recall@1 vient donc entièrement de l’adaptation contrastive, et non d’une capacité de récupération générale déjà présente avant affinage : substituer un grand modèle multimodal à un scoreur lexical sans l’entraîner à la tâche de récupération n’apporte essentiellement rien sur la métrique qui compte le plus pour une réponse à un seul candidat.

Une réserve doit être signalée : les questions synthétiques et le lecteur affiné proviennent de la même famille de modèles, de sorte qu’une partie du gain pourrait refléter les idiosyncrasies du générateur de questions plutôt qu’une amélioration transférable à des questions humaines ; le contrôle direct de ce biais, écrit mais non exécuté faute de budget de calcul, est discuté en section A.5. Ce résultat désigne aussi le correctif direct de l’évaluation A : nous disposons désormais d’un scoreur qui voit l’image de la tuile et pourrait exploiter l’information spatiale que le tuilage guidé préserve — rejouer l’évaluation A avec ce scoreur est l’expérience directement indiquée par l’enchaînement de ces deux résultats (section A.6).

3.4 Composants en aval, coûts et limites : synthèse

Les deux résultats ci-dessus sont solidement étayés. Les deux étapes en aval — le contrôleur de preuves budgété et le système assemblé bout-en-bout — donnent des résultats plus nuancés, analysés en détail en chapitre A, et qui convergent vers un diagnostic unique résumé ici.

Le **contrôleur budgété** (section A.2) atteint la parité avec une sélection top- k statique

après un correctif dont la cause — la frontière d’exploration ne franchissait jamais les limites de document — a été diagnostiquée puis corrigée : la couverture inter-pages passe de 29,7 % à 39,6 %, contre 40,7 % pour la référence. La supériorité n’est pas établie ; les deux politiques sont statistiquement indiscernables sur cet échantillon.

Le **système assemblé** (section A.3) échoue par *abstention* plutôt que par *fabulation* : quatorze prédictions sur vingt déclarent explicitement que les preuves ne suffisent pas, contre quatre qui affirment quelque chose de faux. Le lecteur n’échoue donc pas à *interpréter* les preuves fournies : il constate correctement qu’elles sont insuffisantes. La défaillance est localisée en amont, dans la sélection.

Ces deux observations, ainsi que le résultat nuancé de classement de l’évaluation A, désignent un premier composant, le **score de pertinence du contrôleur**, qui reste un substitut lexical (TF-IDF) alors que l’évaluation B ci-dessus a validé un scoreur nettement supérieur sur exactement cette tâche. Le point d’extension qui permet la substitution est déjà en place dans l’implémentation (section 2.6.4), et la seule barrière est le coût de calcul, un passage avant du modèle par tuile candidate et par question, analysé avec les autres coûts observés en section A.4.

Une seconde limite s’y ajoute, indépendante de la première, et qui tient à la politique plutôt qu’au score. Le contrôleur ne propage que le long de `reading_order`, `links_to` et `continues`. `layout_adjacency` et `section_hierarchy` sont instanciées dans le graphe mais ne sont jamais interrogées, et aucun scoreur ne les rendrait traversables, puisque la fonction de voisinage ne les consulte pas (algorithme 2). Le graphe effectivement parcouru est donc plus proche de deux chaînes cousues ensemble que de la structure à partir de laquelle il est construit. Ce travail établit qu’un tel graphe se construit depuis le DOM, non qu’un contrôleur en tire déjà parti.

Le système ne présente donc pas une défaillance diffuse dont on ignorerait l’origine, mais deux composants validés et deux limites nommées, l’une et l’autre instrumentées. Les limites restantes — portée à Wikipédia, plafond de tuiles, tailles d’échantillon, hyperparamètres non balayés — et les sept directions de travail qui en découlent sont détaillées en sections A.5 et A.6.

Conclusion

Rappel de la thèse

Ce travail est parti d’une observation dont la simplicité est le principal argument : pour une page web, la segmentation dont un système de génération augmentée par récupération visuelle a besoin existe déjà. Le moteur de mise en page du navigateur a calculé la boîte englobante exacte de chaque élément avant qu’un seul pixel ne soit recadré. PixelRAG [37] établit qu’un modèle vision-langage peut récupérer directement sur des tuiles de pixels, mais les découpe à l’aveugle ; MAGE-RAG [45] établit qu’un contrôleur de graphe budgété surpasse une sélection statique, mais suppose les éléments du document déjà constitués en amont de son propre pipeline. Les deux laissent, chacun de son côté, la même pièce sur la table : ce travail les assemble par l’artefact que le premier jette au rendu et que le second n’a jamais eu à construire — le DOM.

Transformer cette observation en système a demandé quatre composants : un tuilage adaptatif aligné sur les frontières d’éléments, un graphe de preuves multigranulaire s’étendant sur plusieurs documents, un lecteur vision-langage adapté par contraste à l’évaluation de pertinence, et un contrôleur budgété décidant à la requête quelles tuiles ouvrir. Le système est implémenté de bout en bout, testé, et évalué sur des pages web réelles plutôt que sur un corpus figé.

Ce que le travail établit

Deux résultats sont solidement étayés et constituent l’apport principal de ce projet.

Le tuilage guidé par le DOM est meilleur et moins cher. La fraction des questions dont la réponse survit dans une unité récupérable passe de 26,4 % avec une grille aveugle à 51,8 % avec nos tuiles. Ce n’est pas un gain de classement mais un gain de *possibilité* : une réponse absente de toute unité est perdue quelle que soit la qualité du récupérateur en aval, de sorte que doubler ce taux double le nombre de questions auxquelles le système peut du tout répondre.

Et ce gain s’obtient à un coût inférieur. Notre tuilage produit cinq fois plus d’unités que la grille, mais chacune est treize fois plus petite : le total des pixels par page est 2,7 fois moindre. L’origine de ce résultat est identifiée et formalisée précisément — la boîte *serrée* d’une balise de texte coulant, mesurée sur l’union des rectangles de ses lignes réellement rendues plutôt que sur sa boîte de bloc CSS pleine largeur, élimine une aire de fond blanc qu’un recadrage naïf aurait soumise en pure perte à l’encodeur visuel — et elle en constitue l’enseignement technique le plus transférable. Comme le nombre de tokens visuels croît avec l’aire de l’image dans la famille de modèles utilisée, ce budget de pixels est un proxy direct du coût d’inférence. Il n’y a donc pas d’arbitrage à faire : la trouvabilité et le coût s’améliorent ensemble.

L’adaptation contrastive du lecteur fonctionne nettement. Sur un partitionnement au niveau de l’article strictement disjoint, et en classant chaque question contre toutes les tuiles de sa page plutôt que contre le vivier miné, le Recall@1 passe de 0,433 à 0,785 et le MRR de 0,578 à 0,856.

La comparaison avec la référence lexicale porte un enseignement qui dépasse notre système. Le modèle vision-langage non adapté obtient un MRR de 0,578, pratiquement identique à celui d’une similarité cosinus TF-IDF (0,579), et un Recall@1 légèrement inférieur — mais un Recall@3 et un Recall@5 légèrement supérieurs : ce n’est pas un déficit général de récupération. Autrement dit : substituer un grand modèle multimodal à un scoreur lexical, sans l’entraîner à la tâche de récupération, n’apporte pas d’avantage net sur le rang de tête, qui est ce qui compte pour une sélection à un seul candidat. Tout le gain de Recall@1 vient de l’adaptation contrastive spécifique.

Ces deux résultats partagent une même racine conceptuelle. Ce travail ne propose ni nouvelle architecture d’encodeur ni mécanisme d’attention inédit : le lecteur affiné reste un Qwen2-VL générique. Ce qui change est *où* passe la frontière de recadrage. En alignant la grille de *patches* que consomme le modèle vision-langage sur les frontières d’éléments du DOM plutôt que sur une géométrie arbitraire, le tuilage garantit qu’une tuile porte un contenu visuel homogène : c’est ce qui rend chaque pixel soumis à l’encodeur pertinent pour la question posée, aussi bien au moment de la récupération non entraînée qu’à celui de l’apprentissage contrastif, dont l’objectif n’a plus à démêler plusieurs sujets mélangés dans une même séquence de *patches*. Le gain de trouvabilité, la réduction du budget de pixels et l’efficacité de l’adaptation contrastive sont ainsi trois manifestations mesurables d’une seule décision de conception, prise avant que l’encodeur ne voie la première tuile.

Le verrou identifié

Les composants en aval n’atteignent pas encore le niveau des deux précédents, et le principal apport analytique de ce rapport est d’avoir localisé précisément pourquoi.

Le contrôleur budgété n’établit pas de supériorité sur une sélection top-*k* statique. Ce résultat a néanmoins produit un enchaînement méthodologique satisfaisant : nous avons formulé une hypothèse causale falsifiable — la frontière d’exploration ne franchissait jamais les limites de document, la relation d’ordre de lecture ne reliant jamais deux pages —, implémenté le correctif déduit de ce diagnostic, et mesuré l’effet prédit. La couverture inter-pages est passée de 29,7 % à 39,6 %, réduisant l’écart avec la référence de dix points à un point. La parité est atteinte, la supériorité non.

L’évaluation bout-en-bout a ensuite montré que le mode de défaillance dominant du système assemblé est l’*abstention explicite* — quatorze prédictions sur vingt — et non la fabulation. Le lecteur n’échoue pas à interpréter les preuves qu’on lui fournit : il constate correctement qu’elles sont insuffisantes. La défaillance est donc située dans la sélection, en amont.

Quatre observations convergent alors vers le même composant. Le contrôleur n’atteint qu’une couverture complète sur un peu plus d’une question sur trois. Douze des quatre-vingt-onze questions ont une couverture nulle, signe que le score ne classe pas correctement même sur la page de départ. Le système assemblé s’abstient parce que les preuves manquent. Et le tuilage structuré n’est pas récompensé par une métrique aveugle à la structure — ce qui explique le seul résultat contraire à nos attentes, le découpage textuel plat devançant les tuiles DOM sur le sous-ensemble strictement apparié.

Ce composant est le **score de pertinence du contrôleur**, qui reste un substitut lexical alors même que l’évaluation du lecteur a validé un scoreur bien supérieur sur exactement cette tâche. Le point d’extension qui permet la substitution est déjà en place dans l’implémentation ; la seule barrière est le coût de calcul, un passage avant du modèle par tuile candidate et par question contre un score lexical essentiellement gratuit.

À ce premier verrou s'en ajoute un second, que le score ne recouvre pas. La politique de parcours n'emprunte que trois des sept relations construites, et l'adjacence de mise en page comme la hiérarchie de sections restent inexploitées quel que soit le scoreur employé, la fonction de voisinage ne les consultant pas. Le graphe est donc établi comme constructible depuis le DOM, pas encore comme exploité.

Il faut dire cela clairement, car c'est une position favorable pour la suite. Le système ne présente pas une défaillance diffuse dont on ignorerait l'origine, mais deux composants validés et deux verrous nommés, localisés, instrumentés, et dont les correctifs sont spécifiés séparément.

Enseignements méthodologiques

Trois enseignements de méthode dépassent le cas particulier de ce système et méritent d'être retenus.

Une métrique de récupération doit être invariante à la taille du vivier. Sur nos données, la grille aveugle atteint un Recall@5 de 0,991 — un score presque parfait qui n'exprime rien d'autre que le fait qu'elle ne propose que cinq candidats par page. Rapporter des Recall bruts pour comparer des stratégies de découpage produit une conclusion inversée. Le rang percentile, dont l'espérance sous classement aléatoire vaut 0,5 quelle que soit la taille du vivier, est la correction nécessaire. Cette précaution nous paraît sous-estimée dans la littérature sur le découpage documentaire.

Le taux d'attachement mérite d'être mesuré explicitement. La proportion de questions dont la réponse survit à l'étape de découpage détermine un plafond que le meilleur récupérateur du monde ne peut pas franchir, et c'est sur cette métrique que l'écart entre les deux stratégies de tuilage est le plus grand. Elle est pourtant absente des protocoles usuels, qui se concentrent sur la qualité du classement en supposant l'attachement acquis.

Une évaluation par composant peut masquer un système qui ne fonctionne pas. Deux de nos quatre évaluations sont franchement positives, et le système assemblé répond correctement à une question sur vingt. Sans l'évaluation bout-en-bout, le rapport aurait pu conclure sur deux succès. C'est elle qui a rendu lisible le mode de défaillance réel, et donc localisé le verrou. Nous retenons de cette expérience qu'une évaluation par composant est nécessaire mais jamais suffisante.

Perspectives

La suite immédiate est déterminée par le diagnostic. Il s'agit de brancher le lecteur validé dans le score de pertinence du contrôleur, à la place du substitut lexical, et de rejouer à l'identique les évaluations du contrôleur et du système assemblé. Une atténuation praticable du coût consiste à adopter l'architecture classique du classement en deux étapes : TF-IDF pour présélectionner une courte liste, le lecteur affiné pour la réordonner. La même substitution appliquée à l'évaluation du tuilage testerait directement si l'avantage observé du texte plat est un artefact du scoreur lexical.

À plus longue portée, deux directions nous paraissent les plus prometteuses. D'une part, faire réellement exploiter au contrôleur les relations du graphe qu'il n'interroge pas encore : remonter au titre de section d'une tuile ouverte, ou atteindre l'en-tête d'un tableau dont une ligne a été retenue, sont des besoins de récupération concrets auxquels le graphe répond

déjà structurellement. D'autre part, sortir du périmètre MediaWiki, ce qui permettrait de séparer empiriquement ce qui relève de la liste de sélecteurs de nettoyage — spécifique par construction — de ce qui relève de la logique de tuilage, qui ne l'est pas.

Mot de la fin

L'affirmation centrale de ce travail résiste à l'examen : le DOM contient bien la segmentation que le RAG visuel reconstruit autrement, et l'exploiter rend davantage de contenu trouvable pour un coût moindre. L'affirmation plus ambitieuse — qu'un graphe de preuves construit sur cette structure, parcouru par un contrôleur budgété, produit un meilleur système de questions-réponses — n'est pas encore démontrée. Elle n'est pas réfutée non plus, car elle attend des expériences identifiées, dont les obstacles sont un budget de calcul et une politique de parcours à étendre, non une inconnue conceptuelle. Nous publions le pipeline complet, y compris les résultats qui ne sont pas allés dans le sens attendu, parce que la localisation du verrou est elle-même un résultat utilisable par qui reprendra ce travail.

Annexe A

Résultats complémentaires

Cette annexe rassemble les résultats que le chapitre 3 résume sans les détailler : le résultat de classement plus nuancé de l'évaluation A (sections A.1.1 et A.1.2), le contrôleur budgété (section A.2), le système assemblé bout-en-bout (section A.3), les coûts de calcul observés (section A.4), les limites du travail (section A.5) et les directions qu'elles ouvrent (section A.6).

A.1 Évaluation A, complément : protocole et classement

A.1.1 Deux contraintes de protocole

Deux caractéristiques du protocole de l'évaluation A conditionnent la lecture des chiffres. **Un plafond de 25 tuiles DOM par page** (section 2.3) borne aussi l'évaluation : aucune tuile au-delà n'a été présentée au modèle générateur, donc aucune question ne peut en dépendre. Sur 17 des 18 pages, le nombre de tuiles DOM est exactement 25 (figure A.1) : la valeur de 24,0 tuiles par page est donc un *plafond atteint*, non une mesure de densité de tuilage réelle, qui serait supérieure. **Les trois conditions couvrent la même région de page** : le protocole calcule une frontière verticale — le bas de la plus basse tuile DOM retenue — et y restreint la grille et les fenêtres de texte, de sorte que les comparaisons de taux d'attachement et de budget de pixels portent bien sur le même contenu.

A.1.2 Résultat 3 : la qualité de classement

Tableau A.1 rapporte le rang percentile (équation (1.1)), sur toutes les paires attribuables puis sur les 107 paires attribuables dans les *trois* conditions simultanément (comparaison strictement appariée).

Trois lectures s'imposent. **L'artefact de vivier est massif** : la grille atteint 0,991 de Recall@5, un score presque parfait qui n'exprime rien d'autre que le fait qu'elle ne propose que 4,9 candidats par page — ce qui valide empiriquement la nécessité du rang percentile. **Le tuilage DOM domine la grille sur la métrique corrigée** : 0,868 contre 0,839 sur toutes les paires, 0,901 contre 0,845 sur le sous-ensemble apparié — un avantage modeste (trois à six points) mais cohérent entre les deux ensembles, qui s'ajoute au doublement du taux d'attachement et à la division par 2,7 du budget de pixels. **Le texte plat devance le tuilage DOM sur les paires appariées** : 0,928 contre 0,901, contrairement à la prémisse partagée par PixelRAG, par MAGE-RAG et par notre propre introduction.

A.1.3 Pourquoi le texte plat reste compétitif

Ce dernier résultat n'est pas un défaut de la condition textuelle : trois facteurs l'expliquent. **Le scoreur n'exploite aucune information spatiale** : TF-IDF, employé dans les trois conditions pour isoler l'effet du découpage, ne sait pas qu'une cellule appartient à une ligne de tableau ou qu'un titre coiffe un paragraphe — il voit un sac de mots, précisément la propriété sur laquelle repose la valeur du tuilage structuré. Le résultat mesure donc le tuilage sur une métrique aveugle à ce qu'il apporte ; la question de savoir si l'information spatiale préservée par des tuiles alignées apporte un gain est traitée séparément par l'évaluation B, qui utilise

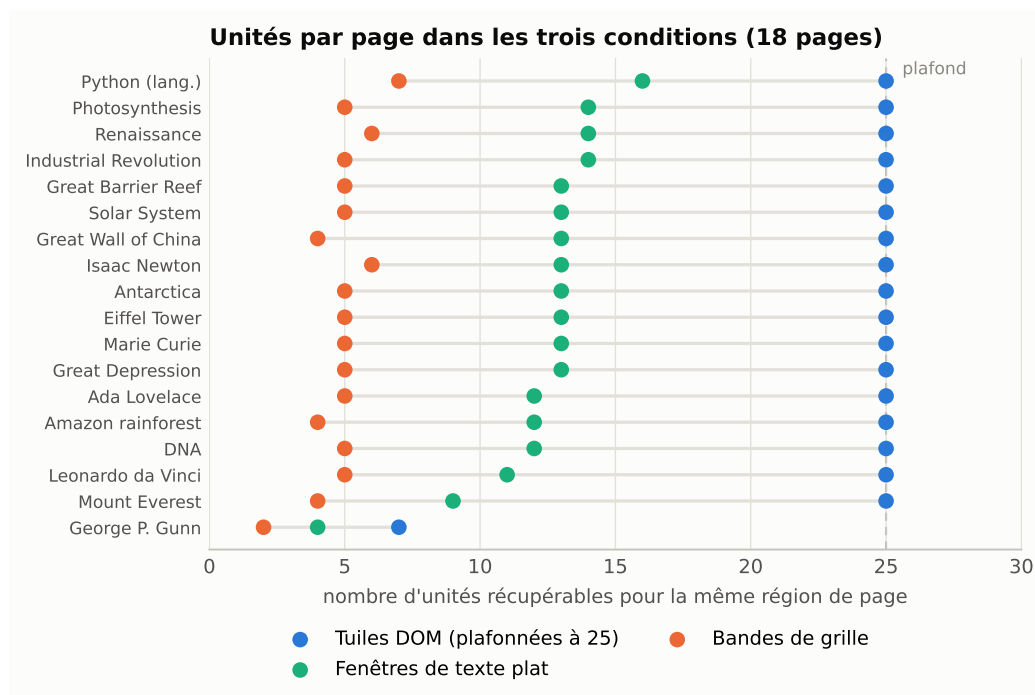


Figure A.1 – Nombre d’unités récupérables produites par chaque condition, page par page, pour la même région de page. Le plafond de 25 tuiles DOM est atteint sur 17 des 18 pages ; seule la page la plus courte (*George_P._Gunn*, 7 tuiles) reste en dessous. La grille aveugle produit systématiquement entre 2 et 7 bandes, soit un ordre de grandeur de moins.

un scoreur capable de la percevoir. **Le critère d’attribution favorise le texte** : une correspondance verbatim est structurellement plus favorable à une condition qui manipule le texte directement qu’à une condition qui relit l’aperçu textuel d’un recadrage d’image — l’essentiel de l’écart de taux d’attachement (56,7 % contre 51,8 %) en découle. **Le texte plat ne porte aucune information visuelle** : un chiffre lu sur un axe, une correspondance ligne-colonne dans un tableau complexe, rien de cela ne subsiste dans du texte extrait — une limite que nos 18 pages, majoritairement textuelles, ne font pas ressortir, mais qui se manifesterait sur un rapport financier ou une notice technique. Cette évaluation établit donc que le texte plat est *compétitif sur ce corpus et sous ce scoreur*, non qu’il est préférable dans l’absolu.

A.2 Évaluation C : contrôleur budgété et sélection top- k statique

A.2.1 Protocole

Cette évaluation mesure la capacité du contrôleur (section 2.6) à rassembler des preuves réparties sur deux documents. Pour chacune des 91 questions inter-pages construites depuis HotpotQA (section 2.8.4), le graphe combiné des deux articles cités est construit, avec recâblage des arêtes `links_to` sur leur point d’entrée réel (section 2.4.6). Le contrôleur et une référence top- k statique sont exécutés sur des scores de pertinence *identiques*, avec le même budget ; seule la politique de sélection diffère. La métrique est la **couverture complète** : les preuves sélectionnées contiennent-elles au moins une tuile positive sur *chacune* des deux pages ? Deux configurations d’amorce sont rapportées : $k_{\text{seed}} = 3$ (valeur par défaut, budget 5) et $k_{\text{seed}} = 5$ (égale au budget).

Tableau A.1 – Qualité de classement des trois conditions, même scoreur TF-IDF, 18 pages et 409 questions. %rang désigne le rang percentile moyen : 0,5 sous classement aléatoire, invariant à la taille du vivier. Les Recall bruts ne sont *pas* comparables entre conditions (section 1.1.1) et sont donnés à titre de diagnostic.

Condition	n	unités/page	R@1	R@5	%rang
<i>Toutes les paires attribuables</i>					
Tuiles DOM	212	24,0	0,547	0,774	0,868
Grille aveugle	108	4,9	0,722	0,991	0,839
Texte plat	232	12,3	0,612	0,875	0,867
<i>Paires appariées (107 attribuables dans les trois conditions)</i>					
Tuiles DOM	107	24,0	0,514	0,841	0,901
Grille aveugle	107	4,9	0,729	0,991	0,845
Texte plat	107	12,3	0,710	0,944	0,928

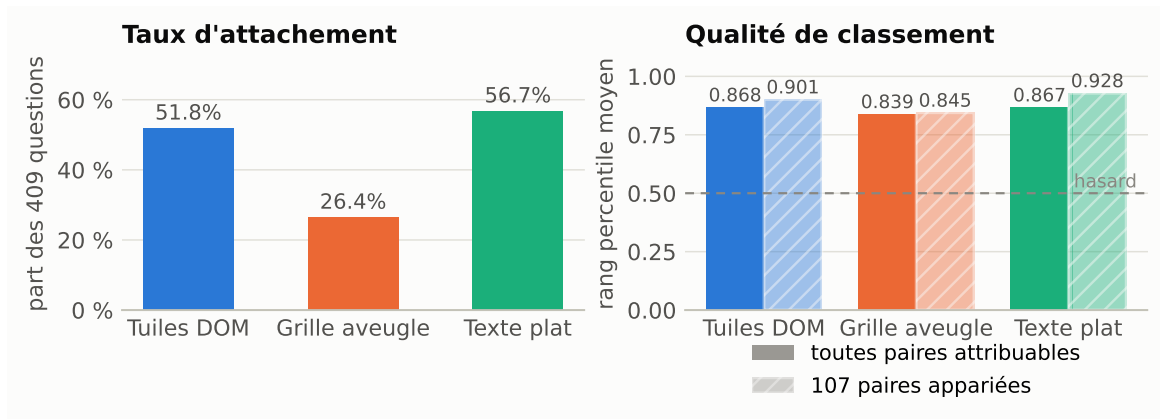


Figure A.2 – À gauche : taux d'attachement. À droite : rang percentile moyen sur toutes les paires attribuables (barres pleines) et sur les 107 paires appariées (barres hachurées). La ligne pointillée marque la valeur attendue sous classement aléatoire.

A.2.2 Diagnostic, correctif et résultat

La première mesure, frontière propagée uniquement le long de `reading_order`, donnait 29,7 % de couverture complète pour le contrôleur contre 40,7 % pour la référence à l'amorce par défaut, et une égalité exacte à $k_{\text{seed}} = 5$ (bloc supérieur du tableau A.2) — attendue, puisqu'à amorce égale au budget le contrôleur ouvre son amorce intégralement et *devient* la référence plutôt que d'y dégénérer, ce qui vérifie la cohérence de l'implémentation. Le déficit à $k_{\text{seed}} = 3$ s'expliquait par l'incapacité de la frontière à franchir une limite de document : `reading_order` ne relie jamais deux pages, si bien que le second article n'était atteignable que si l'amorce y avait déjà placé un nœud.

Après implémentation de l'expansion inter-documents (section 2.6.3), la couverture à l'amorce par défaut passe de 27 à 36 questions sur 91, soit de 29,7 % à 39,6 % — exactement l'effet prédit par le diagnostic, mesuré une seule fois après implémentation plutôt qu'ajusté empiriquement. Ce que le correctif n'établit pas est une supériorité : à l'amorce par défaut le contrôleur reste derrière d'une question, et les intervalles de Wilson à 95 % des deux proportions se recouvrent presque intégralement (30,1 à 49,8 pour le contrôleur, 31,1 à 50,9 pour la référence) : les deux politiques sont statistiquement indiscernables sur cet échantillon.

Tableau A.2 – Fraction des 91 questions inter-pages pour lesquelles les preuves sélectionnées couvrent une tuile positive sur *chacun* des deux articles, avant et après le correctif de frontière inter-documents.

Amorce k_{seed}	Contrôleur budgété	Top- k statique
<i>Avant correctif (<i>reading_order</i> seul)</i>		
3 (budget 5)	29,7 % (27/91)	40,7 % (37/91)
5 (= budget)	40,7 % (37/91)	40,7 % (37/91)
<i>Après correctif (+ <i>links_to</i> et <i>continues</i>)</i>		
3 (budget 5)	39,6 % (36/91)	40,7 % (37/91)
5 (= budget)	40,7 % (37/91)	40,7 % (37/91)

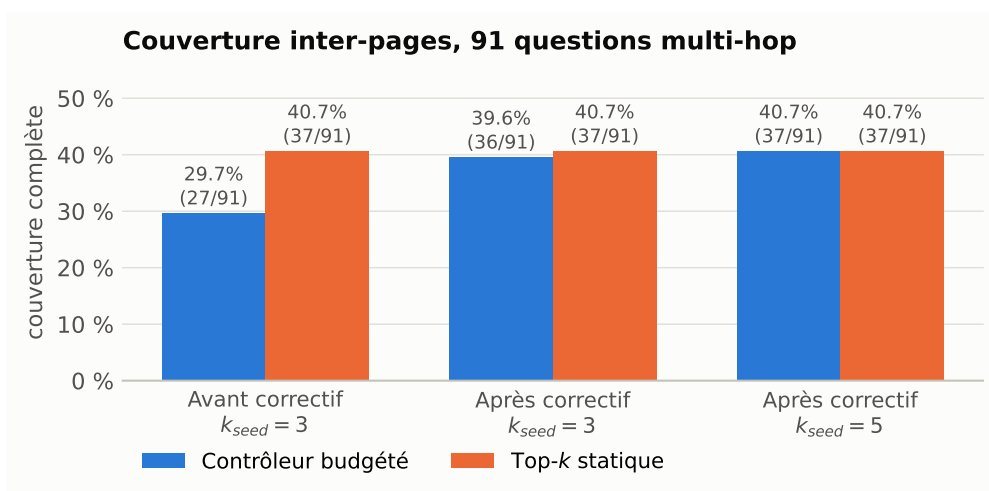


Figure A.3 – Couverture complète inter-pages. Le correctif produit l’effet prédit par le diagnostic : la couverture du contrôleur passe de 27 à 36 questions sur 91 à l’amorce par défaut, réduisant l’écart avec la référence de dix points à un point.

A.2.3 Analyse

Le contrôleur devance la référence sur 7 questions et la suit sur 8 : un échange à peu près équilibré, non un déficit systématique. Il ouvre en moyenne 4,86 nœuds contre 5,00 pour la référence, c’est-à-dire qu’il élague parfois un nœud sous le seuil d’ouverture plutôt que de dépenser son budget — souhaitable en soi, mais sans gain de couverture. Douze questions sur 91 obtiennent une couverture nulle : le problème n’y est pas la propagation de la frontière mais le classement lui-même, qui ne place pas les bonnes tuiles en tête même sur la page de départ — pointant une seconde fois vers le scoreur, non vers la politique. Le correctif de frontière traitait la question de *où* la frontière peut aller, non celle de *à quel point elle juge bien la pertinence* une fois arrivée, et le point d’extension nécessaire (section 2.6.4) est déjà en place.

A.3 Évaluation D : le système assemblé

A.3.1 Protocole et résultat

Cette évaluation mesure le pipeline complet : le contrôleur post-correctif sélectionne les preuves (budget 5, amorce 3), le modèle vision-langage lit chaque tuile ouverte, et un appel de synthèse final produit la réponse. Elle porte sur un tirage aléatoire reproductible (`random.Random(seed=0).sample`) de 20 des 91 questions inter-pages de l’évaluation C —

Tableau A.3 – Précision bout-en-bout du pipeline complet, 20 questions multi-sauts HotpotQA. La taille de l'échantillon ne permet pas d'intervalle de confiance utile ; c'est le mode de défaillance du tableau A.4 qui porte l'information.

Métrique	Valeur
Correspondance exacte	0,050 (1/20)
F_1 sur tokens	0,057

Tableau A.4 – Répartition des 20 prédictions du système assemblé, établie manuellement.

Catégorie	n	Exemple de prédiction
Abstention explicite	14	« Not specified in evidence », « Not found in evidence », « Not determinable from evidence »
Retournement oui/non erroné	1	« No » contre une référence « yes »
Partiellement correcte ($F_1 > 0$)	1	réponse hybride sur « ... Tenerife, Spain ... » contre « Canary Islands, Spain » ($F_1 = 0,133$)
Entité erronée	3	« valuation » contre « business » ; « iMac G5 » contre « keyboard function keys »
Réponse exacte	1	« Blake Shelton »

l'échantillon reste restreint en raison du coût, environ deux minutes par tuile ouverte. Les réponses sont notées par correspondance exacte et F_1 sur tokens après normalisation standard de SQuAD [29].

Une réponse exactement correcte sur vingt, et un F_1 de 0,057 : prise seule, cette mesure indique que le système assemblé n'est pas prêt pour un usage réel. Ce qui la rend informative est la *nature* des erreurs.

A.3.2 Taxonomie des erreurs et analyse

A.3.3 Sous-rappel de preuves plutôt que fabulation

Le mode dominant est l'abstention. Quatorze prédictions sur vingt déclarent explicitement que les preuves ne permettent pas de répondre : le lecteur ne réussit pas à interpréter les preuves qu'on lui donne, il constate correctement qu'elles sont insuffisantes — la défaillance est en amont, dans la sélection. **Cette localisation est cohérente avec l'évaluation C** : sa couverture complète de 39,6 % signifie que sur six questions sur dix, une question multi-sauts est structurellement irrépondable et l'abstention y est la réponse correcte — les deux évaluations mesurent le même goulot par deux voies indépendantes. **La fabulation est marginale** : quatre prédictions sur vingt seulement affirment quelque chose de faux, un comportement bien calibré face à des preuves incomplètes, non garanti pour un système de RAG. **La synthèse échoue aussi lorsque la couverture est complète** : 39,6 % fixe un plancher optimiste que l'exactitude bout-en-bout (une réponse sur vingt) n'atteint pas ; cet écart supplémentaire n'est pas encore diagnostiqué — preuve mal lue, invite sous-spécifiée, ou faiblesse de lecture du modèle de base — et reste à trier avant d'étendre l'échantillon (section A.6). Le retournement oui/non mérite une mention à part : c'est la seule erreur susceptible d'induire un utilisateur en erreur sans qu'il s'en aperçoive, un « non » assuré ne signalant rien de son incertitude.

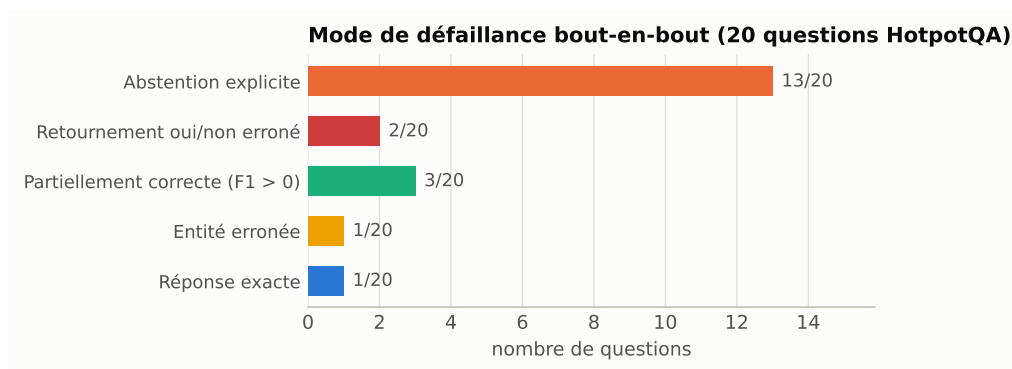


Figure A.4 – Quatorze prédictions sur vingt sont une abstention explicite : le système reconnaît que les preuves rassemblées ne suffisent pas, plutôt que de produire une réponse inventée.

A.4 Coûts de calcul

La génération du jeu de données coûte environ deux minutes par tuile en exécution locale, soit de l'ordre de soixante heures de calcul séquentiel sur 1 784 tuiles ; l'émission concurrente des appels au sein d'une page (section 2.7.4) et le cache sur disque (section 2.7.2) ramènent ce coût à un ordre praticable. L'entraînement contrastif de l'évaluation B a été mené sur une carte A100/H100 de 80 GB louée, lot de 4, deux négatifs difficiles par question, trois époques sur les 1 339 exemples contrastifs ; le *gradient checkpointing* (section 2.5.6) coûte 20 % à 30 % de temps supplémentaire.

C'est le **coût du scoreur appris à l'inférence** qui explique la principale expérience non menée : substituer le lecteur affiné à TF-IDF dans le contrôleur impose un passage avant du modèle par tuile candidate et par question, là où un score lexical est essentiellement gratuit. Sur les 91 questions de l'évaluation C, chacune sur un graphe de plusieurs dizaines de tuiles, cela représente plusieurs milliers de passages avant sur GPU loué — pour une seule configuration d'amorce. Ce coût, non une difficulté d'implémentation, est ce qui a placé l'expérience hors de portée du projet. À l'inverse, le rendu et le tuilage sont négligeables : quelques secondes par page, dominées par l'attente réseau, et la complexité quadratique de l'élagage et du regroupement (section 2.3.3) ne se manifeste jamais aux tailles de page réelles — le tuilage guidé par le DOM est en pratique gratuit, ce qui renforce le résultat de la section 3.2.3 : il ne s'agit pas d'échanger du temps de calcul contre des tokens économisés, mais d'économiser les deux.

A.5 Limites

Les deux limites du contrôleur sont indépendantes. Établies en section 3.4. Le scoreur reste lexical alors que le lecteur affiné de l'évaluation B est une fonction de similarité bien meilleure, et la cause de cette non-substitution est le coût de calcul (section A.4). La politique, elle, n'emprunte que trois des sept relations instanciées (tableau 2.2), de sorte que la richesse du graphe reste inexploitée quel que soit le scoreur employé. Les deux correctifs sont distincts et peuvent être conduits séparément.

Confusion de famille de modèles. Les questions synthétiques et le lecteur affiné proviennent de la même famille Qwen — une contrainte d'infrastructure (section 2.5.3), Ollama n'exposant aucun accès aux poids pour l'affinage. Une partie du gain de l'évaluation B pourrait donc refléter les idiosyncrasies du générateur de questions plutôt qu'une amélioration transférable à des questions humaines. Un contrôle direct — la même mesure de récupération

sur les questions humaines de HotpotQA — est implémenté mais n’a pas été exécuté faute de budget de calcul. Le seul indice disponible est la précision bout-en-bout de 5 % de l’évaluation D, nettement inférieure aux chiffres de l’évaluation B et cohérente avec un déficit de généralisation, sans le démontrer : l’évaluation D change aussi la tâche (pipeline complet, questions inter-pages, mesure de réponse plutôt que de rang).

Le jeu contrastif ne couvre pas tout le corpus. Seize unités de page dont les paires question-réponse ont été générées ne figurent pas dans le jeu contrastif, leur minage de négatifs n’ayant pas abouti lors de la construction. Ces unités portent 378 des 1 717 questions, soit 22 % du total pour 8,6 % des unités. Elles sont en moyenne deux fois et demie plus grandes que le reste du corpus, si bien que la perte frappe préférentiellement les pages longues, celles-là mêmes que ce travail vise. Le lecteur de l’évaluation B est donc affiné sur un sous-ensemble du corpus, et la proportion 80/20 annoncée en section 2.5.7 porte sur ce sous-ensemble. Régénérer le minage sur ces unités serait peu coûteux, le scoreur étant lexical, mais imposerait de réaffiner l’adaptateur et de rejouer l’évaluation B.

Portée limitée aux pages sémantiquement balisées. Tous les résultats quantitatifs portent sur Wikipédia, avec une liste de sélecteurs de nettoyage propre à MediaWiki (section 2.2) ; aucune vérification, même qualitative, n’a été menée hors MediaWiki. La liste de sélecteurs est spécifique ; la logique de tuilage — boîtes englobantes, mesure serrée, fusion, découpe, résolution des chevauchements — ne l’est pas et ne connaît rien de MediaWiki. L’approche n’est pas non plus revendiquée pour les applications monopages hydratées par JavaScript ni les mises en page en soupe de div, le régime que les agents web [6, 43, 44] étudient déjà côté action.

Le plafond de tuiles restreint la portée de l’évaluation A. Le plafond de 25 tuiles (section A.1.1) borne l’évaluation à la portion supérieure des pages longues — non biaisée vers du contenu marginal, les infoboîtes et introductions y étant concentrées, mais partielle. La valeur de 24,0 tuiles par page est un plafond atteint, non une mesure de densité : sans lui, le rapport d’environ cinq entre tuiles DOM et bandes de grille serait plus élevé, dans une direction non mesurée pour le rapport de pixels. Lever ce plafond exigerait de régénérer les paires question-réponse sur l’ensemble du corpus, donc de répéter le coût de la section A.4.

Tailles d’échantillon. Dix-huit pages et 409 questions pour l’évaluation A. 330 questions de validation pour l’évaluation B — la plus solide des quatre. 91 questions pour l’évaluation C, dont les intervalles de Wilson ne justifient qu’une indiscernabilité statistique. 20 questions pour l’évaluation D, qui ne supportent qu’une lecture qualitative du mode de défaillance.

Autres limites. La découpe des éléments démesurés (section 2.3.3) reste indifférente au contenu à l’intérieur d’un seul élément DOM — une version étroite du défaut reproché à la grille. Le parcours de corpus est peu profond (un saut, au plus trois tuiles porteuses de liens, section 2.4.5) ; convient à HotpotQA, pas à une chaîne de trois sauts ou davantage. Le minage de négatifs reste mono-page hors du chemin HotpotQA. Tous les hyperparamètres de l’chapitre B sont des valeurs uniques justifiées mais non balayées, faute de budget de calcul. La relation `semantic_neighbor` et l’ablation de résolution (section 2.8.3) sont implémentées mais non évaluées.

A.6 Directions de travail

Par ordre de bénéfice sur coût : **(1)** brancher le lecteur affiné dans le score de pertinence du contrôleur — le point d’extension existe (section 2.6.4), seule l’exécution est coûteuse ; une atténuation praticable serait un classement en deux étapes, TF-IDF pour présélectionner, le lecteur affiné pour réordonner. **(2)** rejouer l’évaluation A avec ce même scoreur, pour trancher si l’avantage du texte plat est un artefact du scoreur ou un résultat robuste. **(3)** exécuter le contrôle de généralisation déjà écrit (section A.5). **(4)** faire exploiter au contrôleur `section_hierarchy` et `layout_adjacency`, aujourd’hui jamais interrogées. **(5)** miner les négatifs difficiles avec le lecteur lui-même une fois un premier tour d’affinage effectué. **(6)** vérifier qualitativement le tuilage hors MediaWiki, pour séparer ce qui relève du nettoyage de ce qui relève de la logique de tuilage. **(7)** étendre l’évaluation D aux soixante-onze questions restantes de l’évaluation C, une fois triée la cause de l’écart entre couverture de preuves et exactitude bout-en-bout.

Annexe B

Hyperparamètres

Toutes les valeurs référencées dans le chapitre 2 sont rassemblées ici. Chacune est une valeur unique, documentée et centralisée dans un module de configuration ; aucune n’a fait l’objet d’un balayage (section A.5).

Tableau B.1 – Hyperparamètres de la partie hors ligne de la chaîne. Les valeurs en rouge restent à extraire du module de configuration.

Paramètre	Valeur	Référence
<i>Rendu et extraction</i>		
Largeur du <i>viewport</i> de rendu	2 048 px	section 2.2
Hauteur du <i>viewport</i> (sans effet, capture pleine page)	1 000 px	section 2.2
Facteur d’échelle de périphérique	1	section 2.2
Condition d’attente avant capture	<code>networkidle</code>	section 2.2
<i>Tuilage</i>		
Hauteur maximale de tuile (avant découpe)	2 048 px	section 2.3.3
Seuil de mise en tampon (hauteur d’élément)	80 px	section 2.3.3
Tolérance d’imbrication (élagage)	1 px	section 2.3.1
Seuil de chevauchement $\rho_{\min}$	0,15	section 2.3.2
Hauteur de page avant découpage en sections	20 480 px $(10 \times h_{\max})$	section 2.3.5
Budget de hauteur d’une section trop haute	[budget]	section 2.3.5
Tuiles retenues par unité de page (plafond)	25	section 2.3
<i>Graphe</i>		
Tolérance de regroupement en rangées	20 px	section 2.4.1
Tuiles porteuses de liens conservées (k_{liens})	3	section 2.4.5
Pages suivies depuis l’amorce (m)	[?]	section 2.4.6
Seuil de voisinage sémantique (cosinus)	0,2	section 2.4.4
Voisins sémantiques maximum par tuile	3	section 2.4.4
<i>Scoreur lexical TF-IDF</i>		
Plage de n -grammes	[?]	section 2.6.4
Liste de mots vides	[?]	section 2.6.4
Fréquences documentaires minimale et maximale	[?]	section 2.6.4
Pondération sous-linéaire des fréquences	[oui/non]	section 2.6.4
Longueur de l’aperçu textuel par tuile	[caractères]	section 2.3

Tableau B.2 – Hyperparamètres de l’entraînement, du contrôleur en ligne et des conditions de référence.

Paramètre	Valeur	Référence
<i>Entraînement contrastif</i>		
Négatifs difficiles minés par question (K)	2	section 2.5.2
Rang LoRA (r)	16	section 2.5.4
Facteur d’échelle LoRA (α)	32	section 2.5.4
Dropout LoRA	0,05	section 2.5.4
Paramètres entraînables	[?]	section 2.5.4
Optimiseur, taux d’apprentissage, échauffement	[?]	section 2.5.4
Température InfoNCE (τ)	0,05	section 2.5.5
Taille de lot (B)	4	section 2.5.5
Époques	3	section A.4
Proportion du partitionnement (par article)	80/20	section 2.5.7
<i>Contrôleur</i>		
Budget d’ouvertures (B)	5	section 2.6.2
Amorce (k_{seed})	3	section 2.6.2
Seuil d’ouverture (θ_{ouv})	0,05	section 2.6.2
Seuil d’activation (θ_{act})	0,05	section 2.6.2
Appels au modèle concurrents	4	section 2.7.4
<i>Modèles et décodage</i>		
Modèle lecteur (inférence)	qwen3-v1 (Ollama)	section 2.7
Modèle lecteur (affichage)	Qwen2-VL-7B-Instruct	section 2.5.3
Aire d’image minimale du processeur (<code>min_pixels</code>)	[?]	section 2.5.6
Aire d’image maximale du processeur (<code>max_pixels</code>)	[?]	tableau 2.1
Température de décodage	0 (glouton)	section 2.7.2
Fenêtre de contexte	8 192 tokens	section 2.7.2
<i>Références et ablations</i>		
Seuil de découpage en sections, propre à l’évaluation A	51 200 px ($25 \times h_{\text{max}}$, contre $10 \times$ pour le corpus principal)	section 2.8.2
Hauteur de bande de la grille aveugle	1 024 px	section 2.8.2
Taille de fenêtre du texte plat	400 caractères	section 2.8.2
Largeur de bande de la grille aveugle	2 048 px (largeur de rendu)	section 2.8.2
Facteur d’échelle de compression (Lanczos)	0,5	54 section 2.8.3
Graine du tirage aléatoire, sous-échantillon	0	section A.3

Annexe C

Invites utilisées

Les invites sont reproduites telles qu'elles figurent dans le module de configuration. Celles adressées au modèle générateur de questions sont en français, celles de la synthèse finale en anglais — la langue des articles Wikipédia évalués et des réponses de référence de HotpotQA.

C.1 Génération de paires question-réponse

Cette invite accompagne l'image d'une unique tuile (section 2.5.1).

```
1 Tu vois un unique fragment recadré (une "tuile") d'une capture d'écran de
2 page web, pas la page complète.
3
4 Génère UNE question précise et autonome dont la réponse est visible dans
5 cette image, et UNIQUEMENT dans cette image (quelqu'un qui n'a pas vu cette
6 tuile ne pourrait pas deviner la réponse par hasard). Évite les questions
7 vagues ("de quoi parle cette image ?") : vise une question factuelle et
8 spécifique (un nombre, un nom, une date, une définition, une légende...).
9
10 Si l'image ne contient aucune information exploitable (décorative, vide,
11 illisible), réponds exactement: SKIP
12
13 Sinon, réponds STRICTEMENT en JSON, sans texte autour, au format :
14 {"question": "...", "answer": "..."}

```

Trois propriétés de cette invite portent le protocole. L'exigence d'*autonomie* de la question rend la paire utilisable comme exemple positif de récupération : une question dont la réponse serait devinable sans voir la tuile n'apprendrait rien au modèle. La sortie SKIP explicite évite d'avoir à interpréter un refus formulé librement. Et le format JSON strict, sans texte d'encadrement, rend l'analyse syntaxique déterministe.

C.2 Instruction d'encodage PromptEOL

Cette instruction suit l'entrée — question textuelle ou image de tuile, indifféremment — et l'état caché de la dernière couche à la position de son dernier token est pris comme plongement (section 2.5.3).

```
1 \aremplir[instruction PromptEOL exacte, depuis le module de configuration]

```

Son rôle est de contraindre le modèle à comprimer le sens de l'entrée en une position unique ; c'est son application à *l'identique* aux deux modalités qui place question et tuile dans le même espace, sans tête de projection additionnelle.

C.3 Juge de faux négatifs

Ce second filtre, optionnel et non activé pour les jeux de données évalués, est adressé au modèle avec l'image d'une tuile candidate au statut de négatif difficile (section 2.5.2).

```
1 Tu vois un fragment ("tuile") d'une capture d'écran de page web.
2
3 Question : {question}
4 Réponse attendue (trouvée dans une AUTRE tuile de la même page) : {answer}

```

```
5
6 Cette image permet-elle, à elle seule, de retrouver cette même information
7 -- même reformulée (synonyme), ou exprimée visuellement (couleur, logo,
8 forme) plutôt que textuellement ? Réponds STRICTEMENT par un seul mot :
9 OUI ou NON.
```

L'insistance sur les formulations visuelles est le point de cette invite : c'est exactement ce que le filtre lexical par sous-chaîne ne peut pas détecter.

C.4 Lecture d'une tuile ouverte

Adressée au modèle pour chaque tuile que le contrôleur a ouverte (section [2.6.5](#)).

```
1 Question : {query}
2 Extrait uniquement les informations de cette image utiles pour y répondre.
```

C.5 Synthèse de la réponse finale

Adressée au modèle en texte seul, une fois les preuves rassemblées.

```
1 Here are excerpts from a web page, selected as relevant evidence to answer
2 a question.
3
4 Evidence:
5 {evidence}
6
7 Question: {question}
8
9 Answer the question using ONLY this evidence, in English. If the evidence
10 does not allow you to answer, say so explicitly rather than making
11 something up.
```

La dernière phrase de cette invite est directement responsable du mode de défaillance observé en section [A.3.2](#) : elle instruit explicitement le modèle de s'abstenir plutôt que d'inventer. Les quatorze abstentions sur vingt prédictions sont donc un comportement conforme aux instructions face à des preuves incomplètes, et non une défaillance du lecteur.

Annexe D

Organisation du dépôt

Le code source est organisé avec un module par étage du pipeline. Cette correspondance directe entre les sections du chapitre 2 et les modules est délibérée : elle permet de localiser sans ambiguïté l’implémentation de chaque décision de conception décrite.

D.1 Modules du pipeline

Tableau D.1 – Modules du pipeline et sections correspondantes.

Module	Rôle	Section
config.py	constantes centralisées, invites	chapitre B
dom_extraction.py	rendu Playwright, extraction des boîtes englobantes, mesure serrée du texte coulant	section 2.2
tiling.py	élagage, regroupement des chevauchements, appariement, fusion, découpe	section 2.3
sectioning.py	découpage des pages démesurées aux frontières h1/h2	section 2.3.5
graph_builder.py	graphe à deux niveaux, sept relations, export XML	section 2.4
corpus_builder.py	fusion multi-pages, préfixage, points d’entrée	section 2.4.6
vlm_client.py	client vision-langage, cache sur disque, concurrence	section 2.7
qa_generation.py	génération synthétique de paires question-réponse	section 2.5.1
hard_negative_mining.py	minage TF-IDF, double filtre anti-faux-négatif	section 2.5.2
contrastive_dataset.py	jeu de données PyTorch, partitionnement par article	section 2.5.7
lora_finetune.py	tête PromptEOL, LoRA sur ViT et décodeur, perte InfoNCE	section 2.5
evidence_controller.py	machine à états, politique best-first, frontière inter-documents	section 2.6
multihop_qa.py	construction des exemples inter-pages depuis HotpotQA	section 2.8.4
image_compression.py	sous-échantillonnage Lanczos pour l’ablation de résolution	section 2.8.3
visualization.py	superpositions de tuiles, graphes interactifs	—

D.2 Scripts d’expérimentation

Tableau D.2 – Scripts d’expérimentation et évaluations correspondantes.

Script	Rôle	Évaluation
<code>build_dataset.py</code>	pilote le pipeline sur une liste de pages, sauvegarde par unité	section 3.1
<code>build_multihop_dataset.py</code>	construit les exemples HotpotQA, journalise les abandons	section 3.1
<code>grid_baseline.py</code>	référence en grille aveugle, indépendante de <code>tiling.py</code>	A
<code>dom_vs_grid_experiment.py</code>	construit les trois conditions et classe les questions	A
<code>eval_retrieval.py</code>	récupération sur la partie de validation, avec et sans adaptateur	B
<code>eval_retrieval_tfidf.py</code>	référence lexicale sur le même partitionnement	B
<code>eval_multihop_controller.py</code>	couverture inter-pages, contrôleur contre top- k	C
<code>eval_multihop_e2e.py</code>	pipeline complet, correspondance exacte et F_1	D
<code>eval_hotpotqa_reader.py</code>	contrôle de généralisation (<i>écrit, non exécuté</i>)	section A.5
<code>compress_dataset_images.py</code>	produit le dossier miroir en résolution réduite	section 2.8.3
<code>generate_paper_figures.py</code>	superpositions de tuilage des figures 1 et 2.3	—

D.3 Tests

Les tests suivent la séparation par dépendances décrite en section 2.7. Les tests de `tiling`, `graph_builder`, `sectioning`, `hard_negative_mining`, `multihop_qa`, `evidence_controller`, `corpus_builder` et `image_compression` s’exécutent sur des cas synthétiques, sans réseau, sans modèle et sans GPU. Les tests de `dom_extraction` s’appuient sur Playwright et vérifient l’extraction de bout en bout contre une page HTML fournie avec le dépôt, incluant un cas dédié à l’élément flottant partageant sa rangée avec du texte (section 2.2.4).

Annexe E

Protocole de reproduction

E.1 Installation

```
1 python3 -m venv .venv
2 source .venv/bin/activate
3 pip install -r requirements.txt
4 playwright install chromium
5
6 # Backend d'inférence (aucun GPU requis)
7 ollama pull qwen3-vl
```

E.2 Vérification de l'installation

Les tests sans dépendance lourde s'exécutent sur une page locale et ne nécessitent ni réseau ni modèle.

```
1 python3 -m pytest tests/ -x
2 python3 tests/test_pipeline_smoke.py
```

E.3 Reproduction des évaluations

```
1 # Construction du jeu de données (long : ~2 min par tuile en local,
2 # mis en cache sur disque ; reprise automatique par unité de page)
3 python3 -m scripts.build_dataset
4 python3 -m scripts.build_multihop_dataset
5
6 # Évaluation A -- tuilage guidé, grille aveugle, texte plat
7 python3 -m scripts.dom_vs_grid_experiment
8
9 # Évaluation B -- lecteur affiné (nécessite un GPU pour l'entraînement)
10 python3 -m src.lora_finetune --epochs 3 --batch-size 4 \
11 --gradient-checkpointing
12 python3 -m scripts.eval_retrieval
13 python3 -m scripts.eval_retrieval_tfidf
14
15 # Évaluation C -- contrôleur contre top-k statique
16 python3 -m scripts.eval_multihop_controller
17
18 # Évaluation D -- pipeline complet
19 python3 -m scripts.eval_multihop_e2e
```

E.4 Application de démonstration

```
1 uvicorn webapp.backend.main:app --reload
2 # puis ouvrir http://127.0.0.1:8000/
```

Chaque requête recalcule l'intégralité du pipeline sur la page indiquée, sans corpus préconstruit : le temps de réponse va de quelques dizaines de secondes à plusieurs minutes selon la page et le budget du contrôleur (section 2.7).

E.5 Notes de reproductibilité

Trois points conditionnent la reproductibilité exacte des chiffres rapportés. Le **décodage doit être glouton** (température nulle, section 2.7.2) : une température positive rend les appels non reproductibles, avec un symptôme trompeur — la même question peut recevoir une réponse correcte puis une abstention sans aucune différence dans les tuiles ouvertes. La **fenêtre de contexte doit être portée à 8 192 tokens** : à la valeur par défaut de 4 096, un raisonnement long l'épuise avant la réponse, produisant un contenu vide silencieusement interprété comme SKIP. Enfin, les résultats portent en grande partie sur des **pages web vivantes** ; Wikipédia évolue, et les taux de conservation des questions HotpotQA (section 3.1) dériveront avec le temps. Pour l'évaluation A, `scripts/pin_wikipedia_snapshot.py` fige les révisions des dix-huit pages retenues, chargées automatiquement par `dom_vs_grid_experiment.py` avec repli sur le contenu en direct ; les évaluations B, C et D n'en bénéficient pas encore.

Vérification de stabilité. Une réexécution de l'évaluation A sur ce contenu figé (tableau E.1) reproduit le classement et les conclusions qualitatives du tableau A.1 — la grille garde le meilleur R@1 mais le pire taux d'attachement, texte plat et tuilage DOM restent proches et stables. Les écarts, de un à quatre points selon la métrique, sont imputables à la dérive du contenu Wikipédia entre les deux exécutions et ne changent aucune conclusion de la section 3.2.

Tableau E.1 — Évaluation A, chiffres du rapport (tableau A.1, contenu Wikipédia en direct au moment du *run*) contre une réexécution sur le contenu figé par `pin_wikipedia_snapshot.py`. Mêmes dix-huit pages, mêmes 409 questions ; le classement entre conditions et l'écart d'attachement sont préservés.

	<i>n</i>	R@1	Attachement	%rang
<i>Tuiles DOM</i>				
Rapport	212	0,547	51,8 %	0,868
Réexécution, contenu figé	211	0,550	51,6 %	0,869
<i>Grille aveugle</i>				
Rapport	108	0,722	26,4 %	0,839
Réexécution, contenu figé	105	0,762	25,7 %	0,854
<i>Texte plat</i>				
Rapport	232	0,612	56,7 %	0,867
Réexécution, contenu figé	231	0,602	56,5 %	0,864

Considérations éthiques

F.1 Accès aux données

Le constructeur de corpus télécharge une page d’amorce et un nombre borné de pages liées, avec temporisation et reprise plutôt que téléchargement agressif. Un déploiement en production devrait néanmoins préférer l’API REST de Wikimedia ou un *dump* statique au téléchargement en direct ; le prototype ne le fait pas par défaut, et nous le signalons comme une limite de son mode de fonctionnement plutôt que comme un choix défendable à grande échelle.

F.2 Licences

Le texte de Wikipédia est publié sous licence CC BY-SA. Les images qui y sont incorporées portent en revanche des licences hétérogènes, dont certaines non libres. Les captures et les tuiles produites par le pipeline sont donc des œuvres dérivées que nous ne redistribuons pas ; seul le code est publié.

F.3 Statut des données synthétiques

Les questions générées en section 2.5.1 sont produites par un modèle et n’ont pas été vérifiées par un humain. Elles doivent être étiquetées comme telles en cas de diffusion. Cette réserve n’est pas formelle : une question mal formée ou dont la réponse est inexacte introduit du bruit dans l’entraînement contrastif, et le taux de telles questions n’a pas été mesuré.

F.4 Aptitude au déploiement

Le mode de défaillance observé en section A.3.2 est rapporté pour la sécurité du déploiement autant que pour la complétude. L’erreur dominante est l’abstention explicite, la plus sûre des deux pour un système de cette précision. Mais une prédiction sur vingt était une réponse oui/non assurée et fausse — le mode de défaillance le plus susceptible d’induire en erreur un utilisateur qui ne vérifierait pas une affirmation factuelle brève, puisqu’un « non » concis ne signale rien de son incertitude. Un seul cas sur cet échantillon ne permet pas d’estimer un taux, mais suffit à établir que le mode existe et n’est pas détectable par l’utilisateur sans vérification externe.

Nous ne considérons donc pas le système apte au déploiement là où une réponse oui/non erronée porterait des conséquences réelles, et nous rapportons ces résultats précisément pour que cette décision ne se prenne pas sur la seule motivation de la conception.

Bibliographie

- [1] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG : Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024.
- [2] Luiz Bonifacio, Hugo Abonizio, Marzieh Fadaee, and Rodrigo Nogueira. Inpars : Data augmentation for information retrieval using large language models, 2022.
- [3] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.
- [4] Zhuyun Dai, Vincent Y. Zhao, Ji Ma, Yi Luan, Jianmo Ni, Jing Lu, Anton Bakalov, Kelvin Guu, Keith B. Hall, and Ming-Wei Chang. Promptagator : Few-shot dense retrieval from 8 examples, 2022.
- [5] Chao Deng, Jiale Yuan, Pi Bu, Peijie Wang, Zhong-Zhi Li, Jian Xu, Xiao-Hui Li, Yuan Gao, Jun Song, Bo Zheng, and Cheng-Lin Liu. LongDocURL : a comprehensive multi-modal long document benchmark integrating understanding, reasoning, and locating. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1 : Long Papers)*, pages 1135–1159, Vienna, Austria, July 2025. Association for Computational Linguistics. doi : 10.18653/v1/2025.acl-long.57. URL <https://aclanthology.org/2025.acl-long.57/>.
- [6] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web : Towards a generalist agent for the web, 2023. NeurIPS 2023.
- [7] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA : Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.
- [8] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words : Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.
- [9] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global : A graph RAG approach to query-focused summarization, 2024. Microsoft.
- [10] Manuel Faysse, Hugues Sibille, Tony Wu, Bilel Omrani, Gautier Viaud, Céline Hudelot, and Pierre Colombo. Colpali : Efficient document retrieval with vision language models, 2024.
- [11] Alexis Firome. DOM-Graph RAG : DOM-guided adaptive tiling and budgeted online evidence control for multimodal web-page RAG, 2026. Version courte soumise en double aveugle à l’atelier DocInsights 2026 (EMNLP 2026). Le présent rapport en constitue la version longue.

- [12] Tianyu Gao, Xingcheng Yao, and Danqi Chen. SimCSE : Simple contrastive learning of sentence embeddings. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.
- [13] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models : A survey, 2023.
- [14] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora : Low-rank adaptation of large language models, 2021.
- [15] Yupan Huang, Tengchao Lv, Lei Cui, Yutong Lu, and Furu Wei. LayoutLMv3 : Pre-training for document AI with unified text and image masking. In *Proceedings of the 30th ACM International Conference on Multimedia (ACM MM)*, 2022.
- [16] Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2021.
- [17] Ting Jiang, Shaohan Huang, Zhongzhi Luan, Deqing Wang, and Fuzhen Zhuang. Scaling sentence embeddings with large language models. In *Findings of the Association for Computational Linguistics : EMNLP 2024*, pages 3182–3196, 2024.
- [18] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [19] Vladimir Karpukhin, Barlas Öğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.
- [20] Omar Khattab and Matei Zaharia. ColBERT : Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2020.
- [21] Geewook Kim, Teakgyu Hong, Moonbin Yim, JeongYeon Nam, Jinyoung Park, Jinyeong Yim, Wonseok Hwang, Sangdoo Yun, Dongyoon Han, and Seunghyun Park. OCR-free document understanding transformer. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022.
- [22] Kenton Lee, Mandar Joshi, Iulia Turc, Hexiang Hu, Fangyu Liu, Julian Eisenschlos, Urvashi Khandelwal, Peter Shaw, Ming-Wei Chang, and Kristina Toutanova. Pix2Struct : Screenshot parsing as pretraining for visual language understanding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- [23] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.

- [24] Junlong Li, Yiheng Xu, Lei Cui, and Furu Wei. MarkupLM : Pre-training of text and markup language for visually-rich document understanding. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.
- [25] Xueguang Ma, Sheng-Chieh Lin, Minghan Li, Wenhui Chen, and Jimmy Lin. Unifying multimodal retrieval via document screenshot embedding. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.
- [26] Yubo Ma, Yuhang Zang, Liangyu Chen, Meiqi Chen, Yizhu Jiao, Xinze Li, Xinyuan Lu, Ziyu Liu, Yan Ma, Xiaoyi Dong, Pan Zhang, Liangming Pan, Yu-Gang Jiang, Jiaqi Wang, Yixin Cao, and Aixin Sun. Mmlongbench-doc : Benchmarking long-context document understanding with visualizations, 2024.
- [27] Renyi Qu, Ruixuan Tu, and Forrest Sheng Bao. Is semantic chunking worth the computational cost ? In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Findings of the Association for Computational Linguistics : NAACL 2025*, pages 2155–2177, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. doi : 10.18653/v1/2025.findings-naacl.114. URL <https://aclanthology.org/2025.findings-naacl.114/>.
- [28] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.
- [29] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD : 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2016.
- [30] Nils Reimers and Iryna Gurevych. Sentence-BERT : Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.
- [31] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework : BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4) :333–389, 2009.
- [32] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR : A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Advances in Neural Information Processing Systems 34 (NeurIPS), Datasets and Benchmarks Track*, 2021.
- [33] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.
- [34] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding, 2018.
- [35] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.

- [36] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Yang Fan, Kai Dang, Mengfei Du, Xuancheng Ren, Rui Men, Dayiheng Liu, Chang Zhou, Jingren Zhou, and Junyang Lin. Qwen2-VL : Enhancing vision-language model’s perception of the world at any resolution, 2024.
- [37] Yichuan Wang, Zhifei Li, Zirui Wang, Paul Teietche, Lesheng Jin, Matei Zaharia, Joseph E. Gonzalez, and Sewon Min. Pixelrag : Web screenshots beat text for retrieval-augmented generation, 2026. UC Berkeley / BAIR / Berkeley NLP. Code : github.com/StarTrail-org/PixelRAG.
- [38] Zilong Wang, Yiheng Xu, Lei Cui, Jingbo Shang, and Furu Wei. LayoutReader : Pre-training of text and layout for reading order detection. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4735–4744, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi : 10.18653/v1/2021.emnlp-main.389. URL <https://aclanthology.org/2021.emnlp-main.389/>.
- [39] Yang Xu, Yiheng Xu, Tengchao Lv, Lei Cui, Furu Wei, Guoxin Wang, Yijuan Lu, Dinei Florencio, Cha Zhang, Wanxiang Che, Min Zhang, and Lidong Zhou. LayoutLMv2 : Multi-modal pre-training for visually-rich document understanding. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.
- [40] Yiheng Xu, Minghao Li, Lei Cui, Shaohan Huang, Furu Wei, and Ming Zhou. LayoutLM : Pre-training of text and layout for document image understanding. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, 2020.
- [41] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA : A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.
- [42] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct : Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [43] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. GPT-4V(ision) is a generalist web agent, if grounded, 2024. ICML 2024.
- [44] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xiaoyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena : A realistic web environment for building autonomous agents, 2023. ICLR 2024.
- [45] Yilong Zuo, Xunkai Li, Jing Yuan, Qiangqiang Dai, Hongchao Qin, and Ronghua Li. MAGE-RAG : Multigranular adaptive graph evidence for agentic multimodal RAG in long-document QA, 2026. Code : github.com/laonuo2004/MAGE-RAG.